Taiwan Cruise — 車輛操控開發 Prompt 包 v1.5

- **用途：**交給 Codex 分階段開發 three.js 賽車遊戲的「車輛操控」原型（不含場景）
- **手感目標：**Forza Horizon 預設輔助（ABS / TCS / STM 開、自動排檔）
- **內容：**AGENTS.md、Phase 1–7 任務 prompt、手感調校對照表、輪胎與輔助系統的 TypeScript 參考實作（已通過型別檢查與 22 項單元測試）

> **v1.1 修訂（參考實作經模擬驗證後）**
>
> 1. Phase 3 側向力公式：`MF_lat` 的參數應為 `s × tan(peakSlipAngle)`，v1.0 多套了一層 `atan`。
> 2. Phase 3 `gripUsage` 改為「正規化合成滑動量 s」（< 1 抓地、> 1 滑動）。v1.0 的定義永遠 ≤ 1，無法拿來判斷是否在滑。
> 3. Phase 3 車輪積分改為「線性化隱式」。低速時輪胎剛性極高，顯式積分在 480 Hz 下仍會發散。
> 4. Phase 3 低速駐車改為「錨點彈簧-阻尼」。純阻尼擋不住坡道重力，車會以等速慢慢滑。
> 5. Phase 5 TCS 增益改為 kp 0.25 / ki 5。v1.0 的 kp 2 / ki 10 會在全砍與恢復之間循環，加速反而比沒開 TCS 還差。

> **v1.2 修訂（全文審查）**
>
> 1. **懸吊 restLength 0.35 → 0.42 m**。原本的數值與「CoM 離地 0.50 m」矛盾：靜態壓縮約 0.10 m，用 0.35 m 車身會比規格低約 7 cm。
> 2. **車身 collider 必須往上偏移**（中心在 CoM 上方 0.35 m）。原本沒有指定，1.4 m 高的方塊若以 CoM 為中心，底部會插進地面 20 cm。
> 3. **車身剛體要關閉休眠**（setCanSleep(false)），並鎖定 Rapier 版本。否則低速時剛體會睡著，輪胎力施加不上去。
> 4. **後輪最大煞車扭力 1200 → 900 N·m**（前後比 65% → 71%）。1 g 煞車時前後負載約 71/29，原本的比例會讓後輪先鎖死，sim 模式（無 ABS）下重煞會打滑旋轉。
> 5. **自動排檔加入遲滯**。原本 kick-down 的門檻（redline − 300 = 6500）剛好等於全油門升檔點，會在兩檔之間來回跳。
> 6. **倒檔改為「停住後重新踩煞車」才進入**，並補上「回到 D 檔」的規則。原本「停住後持續踩煞車 0.25 s 就進倒檔」，會讓在坡道上踩住煞車的車開始倒退，與「15% 坡道踩煞車不滑動」的驗收互相矛盾。
> 7. **STM 的目標 yaw 改用駕駛意圖輪角**（轉向輔助之前）。原本用含自動反打的輪角，甩尾回正時會煞錯邊，把車甩到另一側。
> 8. **轉向輔助的窗口只限制幅度**，不會主動把輪子轉到另一側；反打全部交給 countersteer，避免兩者疊加反打過頭。
> 9. **自動反打門檻改為 peakSlipAngle × 1.1 ≈ 9°**。v1.1 的 6° 仍低於後輪峰值滑動角 8°，正常極限過彎時就會介入。
> 10. **新增低速橫向阻尼**。模擬發現停車後若有些微側向速度，輪胎鬆弛模型會像阻尼很小的 6 Hz 彈簧，第 2–3 秒仍有 0.011 m/s 晃動，超過驗收值 0.01 m/s。

11. **轉向平滑只做一次**。原本 Phase 4 的速率限制（3 /s）和 Phase 6 的鍵盤漸變重疊，鍵盤設定會失效，手把也會多一層延遲。現在統一在 Steering.ts 依裝置設定。
12. **sport 預設的「TCS 低」補上具體數值**；ABS 的 brakeCmd 明確定義為「玩家 + STM」的總煞車需求。
13. 參考程式碼的 package.json 補上 typecheck / test 指令。

v1.3 修訂（第二次審查）

1. **差速器規格重寫**。原本 LSD 寫成「固定扭力 × sign(轉速差)」，兩輪轉速接近時正負號每步翻轉，會高頻抖動；上限寫成「…」沒有定義；locked 用「顯式強阻尼耦合」，阻尼一大就發散。現在改成「最多轉移到兩輪同速為止」，並新增 Drivetrain.ts 參考實作與測試。
2. **車輪子步進拆成單步 API（substepWheel）**。原本 stepWheel 會把一個輪子的子步進一次跑完，差速器沒辦法在子步進之間耦合左右輪。驅動輪改用 stepDrivenAxle。
3. **STM 的 limitRatio 0.85 → 1.0**。0.85 時，極限定圓的 yaw 只要比穩態多 10%（進彎時很常見）就會觸發煞車，讓車在極限邊緣被拉住。
4. **VehicleState 補齊**駐車錨點、換檔計時器、倒檔狀態、離合器、輸入平滑狀態。少了這些，replay 會對不上。

v1.4 修訂（第三次審查）

1. **懸吊阻尼速度改用車身掛載點速度投影**。原本用 (x − x_prev) / dt，壓縮量不連續變化時（開上路緣或台階、按 R 重置、斜坡邊緣）會產生假的速度尖峰，例如一步內 0 → 0.1 m，在 120 Hz 換算成 12 m/s，阻尼力瞬間約 2.5 萬 N，車會被彈起來。
2. **自動排檔與起步離合器補上精確規則**：換檔判斷一律用「車輪換算轉速」，不用引擎轉速；起步時離合器把引擎維持在 launchRpm。原本起步半離合時引擎可能空轉衝過 6500 rpm，車還沒動就升到 2 檔。
3. **新增整車縱向估算測試**，確認預設 preset 和驗收目標一致：0→100 = 6.17 s、極速 253 km/h、100→0 = 36.9 m，三項都在範圍內。
4. 小修：Phase 3 暫時驅動的扭力改為「後軸總扭力 + open 差速器」；TCS 在倒車時要把滑動率取反；對照表的「靜止微晃」補上低速橫向阻尼；已知限制補上 floating origin 時要平移駐車錨點。

v1.5 修訂（第四次審查）

1. **手煞車扭力不經過 ABS**。v1.2 把 ABS 的輸入定義為「玩家煞車 + STM」，但沒有寫明手煞車要繞過 ABS。若手煞車也進 ABS，ABS 一偵測到後輪鎖死就會放開，手煞車甩尾做不出來。
2. **STM 在拉手煞車時暫停，放開後平滑恢復**。原本 STM 會立刻把手煞車甩尾拉回，和「手煞車進彎可以甩尾」的驗收互相矛盾。
3. **Ackermann 的處理順序寫清楚**：Steering.ts 只輸出一個中心輪角，轉向輔助修正後，最後才依 Ackermann 分成左右輪角。原本 Phase 4 說 Steering.ts 做 Ackermann，Phase 5 又說轉向輔助在它之後，兩邊對不上。
4. **測試場的「外傾 10° 彎道」改為「外側較高、往內傾斜 10° 的超高彎道」**，避免被做成往外傾斜的反向坡。

> 5. 修正 v1.4 第 1 點的說明：用模擬確認自由落下時每一步的壓縮量變化其實不大，真正會產生假尖峰的是壓縮量不連續的情況。Phase 2 新增的落地測試在預設參數下做得到：從 0.5 m 落下，著地第一步約 1.8 倍靜態負載，之後不會再離地。

使用方式

1. 建立新的 repo，把下方的 **AGENTS.md** 放到根目錄。
2. 把附帶的 `handling-reference.zip`（第五章的程式碼）解壓縮，得到 `reference/` 資料夾，放進 repo 根目錄，Codex 實作 Phase 3、Phase 4（差速器）和 Phase 5 時要對照。
3. **一次只貼一個 Phase** 給 Codex，自己試開確認後，再貼下一個。
4. Phase 1–4 先求物理正確（數值合理、不抖、測試通過），Phase 5 之後才開始調「感覺」。

一、AGENTS.md（放在 repo 根目錄）

```
# 專案: Taiwan Cruise — 車輛操控原型

## 目標
使用 three.js + Rapier 開發一個瀏覽器賽車操控原型，手感目標為「Forza Horizon 預設輔助（ABS/TCS/STM 開、自動排檔）」:
好上手、可預期、不會突然失控，但仍能感受到重量轉移、抓地極限、推頭/甩尾。
本階段「只做車子與測試場」，不做真實地景。

## 技術棧（不得自行更換）
- TypeScript (strict) + Vite
- three.js（WebGLRenderer 即可，之後才換 WebGPU）
- @dimforge/rapier3d-compat（package.json 鎖定精確版本，不用 ^）：只用於「車身剛體 + 世界碰撞 + raycast」
  - 車身剛體必須 setCanSleep(false)，否則低速時會休眠，輪胎力施加不上去
  - 不使用 Rapier 內建的 DynamicRayCastVehicleController，懸吊/輪胎/動力系統全部自己寫
- Tweakpane: 即時調參
- Vitest: 單元測試
- 不使用 React / R3F

## 參考實作
reference/ 內有輪胎、車輪積分、駐車力、差速器、ABS/TCS/STM/轉向輔助的參考程式碼與測試。
實作 src/vehicle/ 時，公式、符號約定、數值穩定性處理必須與參考實作一致；可以重構，但不得改變行為。

## 單位與座標
- 全部使用 SI 單位: m、kg、s、N、N·m、rad、rad/s。UI 顯示才轉 km/h、度、rpm。
- 右手座標，Y 軸朝上。車身 local: 前方 = -Z、右方 = +X、上方 = +Y（與 three.js 相機慣例一致）。
- 車身 local 原點 = 質心（CoM）。
- 輪角 > 0 = 向右; yawRateRight = −ω_y（> 0 表示車頭往右轉）。

## 架構規則
1. 模擬與渲染分離:
   - 物理固定步長 PHYSICS_HZ = 120（可設定為 240），使用 accumulator; 每幀最多跑 8 個步長，避免 spiral of death。
   - 渲染端用前後兩個物理狀態做插值（位置 lerp、旋轉 slerp）。
2. 車輛模擬是純邏輯模組（src/vehicle/**），不得 import three.js; 只能依賴一個小型的 Vec3/Quat 數學工具，以及一個 PhysicsAdapter 介面（包裝 Rapier）。這樣之後才能拿到伺服器上跑，也方便寫單元測試。
3. 每一步的輸入是一個可序列化的 VehicleInput: { steer: -1..1, throttle: 0..1, brake: 0..1, handbrake: 0..1, shiftUp: bool, shiftDown: bool }。
4. 車輛狀態必須能序列化成 VehicleState，供 replay 與未來的多人同步使用。必須包含所有會影響下一步的內部狀態:
```

車身位置/旋轉/線速度/角速度、每輪 ω、輪胎鬆弛狀態（κ、α）、懸吊壓縮量（含上一步的值）、駐車錨點、
引擎 rpm、檔位、換檔計時器、倒檔狀態（含「停住後煞車是否放開過」）、離合器結合度、輸入平滑狀態、各輔助系統的積分器與濾波器。
Replay 測試要能抓出漏掉的狀態。
5. 所有車輛參數集中於 /src/vehicle/presets/*.json，程式碼中不得出現魔術數字。Tweakpane 可即時修改，並支援匯出/匯入 JSON。
6. 決定性（determinism）：同一個初始狀態 + 同一串輸入，在同一台機器、同一個瀏覽器上必須得到相同結果。不得在模擬中使用 Math.random 或 Date.now。
（跨瀏覽器的 Math.sin/atan 等結果可能有末位差異，所以多人連線不能依賴決定性，必須靠狀態同步。）
7. 每完成一個 Phase: npm run typecheck、npm run test、npm run build 必須全部通過。

目錄
src/
  core/   loop.ts（固定步長、插值）、input/（鍵盤、手把）、math/
  physics/   PhysicsAdapter.ts、RapierAdapter.ts
  vehicle/   Vehicle.ts、Suspension.ts、Tire.ts、Wheel.ts、Hold.ts、Drivetrain.ts、Engine.ts、Gearbox.ts、Differential.ts、Brakes.ts、Aero.ts、Steering.ts、assists/（PI.ts、abs.ts、tcs.ts、stm.ts、steeringAssist.ts）、presets/
  render/   VehicleView.ts（方塊車身 + 圓柱輪子）、TestTrack.ts
  camera/   ChaseCamera.ts、HoodCamera.ts
  debug/   Telemetry.ts、TuningPanel.ts、Graphs.ts、Replay.ts
reference/   參考實作（唯讀）
tests/

預設車輛 presets/default.json（初始值，之後調校）
- 質量 1350 kg；車身尺寸 1.8(W) × 1.4(H) × 4.4(L) m
- 慣性主軸（local）：Ixx(pitch)=2400、Iyy(yaw)=2550、Izz(roll)=600 kg·m²
- 軸距 2.65 m；前軸在 CoM 前方 1.27 m，後軸在 CoM 後方 1.38 m（前 52% / 後 48%）
- 輪距 前 1.55 / 後 1.55 m；懸吊掛載點位於 CoM 上方 0.15 m
- CoM 離地高度（靜止時）約 0.50 m
- 輪胎半徑 0.33 m；車輪轉動慣量 1.0 kg·m²（含煞車碟）
- 車身 collider：方塊 1.8 × 1.4 × 4.4 m，中心位於 CoM 上方 0.35 m（底部離地約 0.15 m）
- 懸吊：restLength 0.42 m、行程 0.20 m（bump stop 在壓縮 0.18 m 開始，剛性 ×10）
  - 檢查：掛載點離地 0.65 m − 輪胎半徑 0.33 m = 0.32 m；restLength 0.42 − 靜態壓縮約 0.10 = 0.32 ✓
  - 彈簧 k：前 34000、後 32000 N/m
  - 阻尼：bump 前 2100 / 後 2000、rebound 前 3400 / 後 3200 N·s/m
  - 防傾桿：前 15000、後 9000 N/m
- 輪胎：
  - 縱向 peakSlipRatio 0.10、C_long 1.5、μ_long 1.10
  - 側向 peakSlipAngle 8°、C_lat 1.30、μ_lat 1.05
  - 負載敏感度 loadSensitivity 0.10、nominalLoad 3300 N
  - relaxationLength 側向 0.35 m、縱向 0.10 m
- 駐車錨點：engageSpeed 0.3、releaseSpeed 0.5 m/s、頻率 3 Hz、阻尼比 1.0
- 低速橫向阻尼：fadeStart 1、fadeEnd 3 m/s、頻率 6 Hz、阻尼比 0.7
- 引擎：idle 900、redline 6800、limiter 7000 rpm；引擎慣量 0.20 kg·m²
  - 扭力曲線 rpm:N·m = 1000:200, 2000:300, 3000:350, 4000:350, 5000:340, 6000:310, 7000:260
  - 引擎煞車扭力 = 30 + 0.006 × rpm（N·m，油門 0 時）
- 變速箱：檔比 [3.40, 2.20, 1.60, 1.25, 1.00, 0.82]、倒檔 3.20、終傳 3.70、換檔時間 0.15 s、傳動效率 0.90
- 驅動：RWD（支援 FWD / RWD / AWD；AWD 前後扭力分配預設 40/60）
- 差速器：LSD（type: open | lsd | locked），lsd 預載 50 N·m、鎖定係數 0.25
- 煞車：每輪最大扭力 前 2200 / 後 900 N·m（前 71%，對應 1 g 煞車時的前後負載）；手煞車後輪 2500 N·m
- 空力：CdA 0.72 m²、ClA（下壓）0.30 m²（前 45% / 後 55%）、空氣密度 1.225
- 滾動阻力係數 0.012
- 轉向：低速最大輪角 33°、Ackermann 0.6
  - 輸入平滑（只在 Steering.ts 做一次，依裝置）：鍵盤轉入 0→1 需 0.18 s（低速）~ 0.35 s（高速）、回正 0.12 s；手把轉入 0.08 s、回正 0.06 s；方向盤不平滑
- 輔助：ABS kp 1.5 / ki 20；TCS kp 0.25 / ki 5；STM kp 2500 / ki 4000 / limitRatio 1.0（拉手煞車時暫停）；手煞車不經過 ABS；轉向輔助 strength 0.7、反打 gain 0.6、門檻 9°（= peakSlipAngle × 1.1）

手感驗收目標（使用 "forza_normal" 輔助預設）
- 0→100 km/h：6.0–7.5 s；極速 230–260 km/h

```
- 100→0 km/h 煞車：36–42 m，ABS 啟動時方向仍可控
- 半徑 50 m 定圓：穩態側向加速度 0.95–1.05 g
- 靜止不動 2 秒後，車身速度 < 0.01 m/s；在 15% 坡道踩住煞車 5 秒位移 < 2 cm
- 120 Hz 與 240 Hz 下行為差異小（0→100 誤差 < 3%）
- 任何輸入組合都不會出現 NaN、車身爆飛或高頻抖動
```

二、分階段任務 Prompt

Phase 1：專案骨架、固定步長迴圈、測試場、剛體

```
請先閱讀 AGENTS.md，再完成 Phase 1。

1. 用 Vite + TS strict 建立專案，安裝 three、@dimforge/rapier3d-compat、tweakpane、vitest。
   建立 scripts：dev、build、typecheck、test。
2. core/loop.ts：固定步長 accumulator（PHYSICS_HZ 可設定），每幀最多 8 步。
   提供 alpha 給渲染插值。分頁切走後回來時，不得一次補跑大量步長。
3. physics/PhysicsAdapter.ts 定義介面：
   createChassis(desc) / castRay(origin, dir, maxDist, excludeBody) → { hit, point, normal,
distance, colliderUserData }
   applyImpulseAtPoint、applyTorqueImpulse、getLinVel、getAngVel、getPose、setPose、step(dt)。
   RapierAdapter.ts 實作此介面：
   - 車身 collider 的 density 設為 0，用 setAdditionalMassProperties 精確指定質量、CoM、主慣性（依
preset）。
   - 車身 collider 中心放在 CoM 上方 0.35 m（依 preset），不要以 CoM 為中心，否則底部會插進地面。
   - 車身剛體 setCanSleep(false)。
   - 「接觸點速度」自行計算：v + ω × (p − com_world)。
   - 外力一律以「力 × dt 的衝量」在每個物理步長施加。
4. render/TestTrack.ts（只用基本幾何體，地面加上格線貼圖以便感受速度）：
   - 2 km × 2 km 平地
   - 半徑 50 m 定圓（地上畫線）
   - 間距 20 m 的繞錐 10 支
   - 一段 8% 與一段 15% 坡道
   - 連續髮夾彎 4 個（內半徑 12 m、帶 8% 坡度，模擬台14甲）
   - 小顛簸路段（正弦起伏，振幅 5 cm、波長 3 m）
   - 低摩擦區（collider userData 帶 surfaceGrip = 0.7，代表濕地）
   - 超高彎道一個：外側較高、路面往彎道內側傾斜 10°（不是往外傾斜的反向坡）
   每個 collider 的 userData 帶 { surfaceGrip, surfaceType }。
5. render/VehicleView.ts：方塊車身 + 4 個圓柱輪子（之後會換 GLB，所以要保留 setModel 介面）。
6. 此階段車輛先只做剛體，不做懸吊。按 R 可重置到起點。
驗收：可以跑起來、看到測試場；車身方塊從 1 m 高落下會穩定停住；typecheck/test/build 通過。
```

Phase 2：Raycast 懸吊

```
請先閱讀 AGENTS.md，完成 Phase 2：懸吊（Suspension.ts）。

每個物理步長、每個輪子：
1. 由掛載點（車身 local → world）沿車身 -Y 方向 raycast，長度 = restLength + wheelRadius。
   沒有打到地面 → 視為離地：懸吊伸長到 restLength，Fz = 0。
2. 壓縮量 x = restLength − (hitDistance − wheelRadius)，clamp 到 [0, travel]。
   壓縮速度 v_c = −dot(掛載點的車身速度，車身 up)（車身往下壓為正；地面靜止）。
   不可用 (x − x_prev) / dt：車輪從離地到著地的那一步，x 可能從 0 跳到 0.15 m，換算成 18 m/s，阻尼力瞬間超過
3 萬 N，車會被彈起來。
3. 懸吊力：F = k·x + c·v_c（v_c > 0 使用 bump 阻尼，< 0 使用 rebound 阻尼）。
   x > bumpStopStart 時，額外加上 bump stop 力。F 最小為 0（不能把車往下拉）。
```

4. 防傾桿：F_arb = k_arb × (x_left − x_right)；壓縮較多的一側加上 F_arb，另一側減去 F_arb（結果仍 clamp ≥ 0）。
5. 施力方向 = 接觸面法線（預設），施力點 = 接觸點。
6. Fz（輪胎法向負載）= 最終懸吊力，提供給下一階段的輪胎模型。
7. VehicleView 中，輪子的視覺高度要跟著懸吊位置移動。
Debug：用 Tweakpane 即時調整 k、c、arb；畫面上以 4 條長條顯示每輪 Fz（N）與壓縮量。

單元測試：
- 靜止平衡：車放在平地 3 秒後，4 輪 Fz 總和 = m·g ±1%，前後比例 ≈ 52/48 ±2%。
- 從 0.3 m 高度落下：不得穿地，1.5 秒內停止震盪（車身垂直速度 < 0.02 m/s）。
- 120 Hz 與 240 Hz 的穩態壓縮量差異 < 2%。
- 從 0.5 m 高度平落：著地那一步單輪懸吊力 < 4 × 靜態負載，不得彈離地面（預設參數模擬約 1.8 倍；之後壓到 bump stop 時約 5 倍屬正常）。

Phase 3：輪胎模型（手感核心，要精確實作）

請先閱讀 AGENTS.md，完成 Phase 3：Tire.ts、Wheel.ts、Hold.ts。
請以 reference/src/vehicle/Tire.ts、Wheel.ts、Hold.ts 為準，並把 reference/tests 裡相關的測試搬進 tests/。

座標：在接觸面上，用車輪朝向投影出 wheelForward（滾動方向）與 wheelRight，兩者都要投影到接觸平面並正規化。
v_contact = 接觸點的世界速度；v_x = dot(v_contact, wheelForward)；v_y = dot(v_contact, wheelRight)。

1. 滑動率與滑動角
   - 縱向滑動率 κ = (ω·r − v_x) / max(|v_x|, |ω·r|, V_EPS)，V_EPS = 0.5 m/s
   - 穩態滑動角 α_ss = atan2(v_y, max(|v_x|, V_EPS))，往右滑為正
   - 鬆弛長度：α += (α_ss − α) × (1 − exp(−(|v_x| + V_RELAX_MIN) / σ_lat × dt))，κ 同樣處理（用 σ_long）
     V_RELAX_MIN = 1.0 m/s。使用解析解形式，dt 再大也不會發散。

2. 曲線（正規化 Magic Formula，E = 0）
   MF_n(s; C) = sin(C · atan(tan(π / 2C) · s))，s = 1 時為峰值 1.0
   C 必須在 (1, 2) 之間。C_lat = 1.30 時完全滑動後仍保有約 89% 抓地力，這就是 Forza 那種「寬容」的關鍵。

3. 負載敏感度
   loadFactor = clamp(1 − loadSensitivity × (Fz / nominalLoad − 1), 0.3, 1.5)
   μ_eff = μ × surfaceGrip × loadFactor

4. 合成滑動（摩擦圓），使用正規化滑動法：
   s_x = κ / peakSlipRatio，s_y = tan(α) / tan(peakSlipAngle)，s = √(s_x² + s_y²)
   若 s < 1e-9 → F = 0
   否則：
     F_x = μ_eff_long × Fz × MF_n(s; C_long) × (s_x / s)
     F_y = −μ_eff_lat × Fz × MF_n(s; C_lat) × (s_y / s)
   gripUsage = s（< 1 抓地區、= 1 峰值、> 1 滑動中），提供給遙測、TCS、ABS、胎痕使用。

5. 車輪轉動（Wheel.ts）— 必須使用線性化隱式積分
   I·dω/dt = T_drive − r·F_x(ω) − T_resist
   - 每個物理步長跑 WHEEL_SUBSTEPS = 4 次子步進；子步進期間車身速度固定，每個子步進都重算輪胎力，
     最後把平均的 F_x、F_y 當作衝量施加給車身。
   - 單一子步進寫成 substepWheel；非驅動輪可用 stepWheel 一次跑完，驅動輪必須用 Drivetrain.ts 的 stepDrivenAxle
     （每個子步進先分配差速器扭力，再推進兩輪）。
   - 低速時 ∂F_x/∂ω 極大，顯式積分會發散。所以每個子步進：
     K = max(0, ∂F_x/∂ω)（數值微分，eps = 1e-3 × max(1, |ω|)）
     ω_free = ω + h·(T_drive − r·F_x0) / (I + h·r·K)
     resist = h·(T_brake + T_rolling) / (I + h·r·K)
     |ω_free| ≤ resist → ω = 0（鎖死，不跨零震盪）；否則 ω = ω_free − sign(ω_free)·resist
   - 輪胎力施力點 = 接觸點；方向 = F_x·wheelForward + F_y·wheelRight。

```
6. 低速駐車（Hold.ts）— 錨點彈簧-阻尼
   純阻尼擋不住坡道重力分量，車會等速慢慢滑，所以要用錨點：
   - 條件：throttle < 0.05 且 brake（含手煞車）> 0.05，且接觸點平面速度 < 0.3 m/s → 記錄接觸點為錨點
   - 力 = −k·(p − anchor) − c·v（只取接觸平面分量），k = m_corner·(2π·3Hz)², c =
2·1.0·m_corner·(2π·3Hz)
   - |力| > μ_eff·Fz：力限制在 μ_eff·Fz，錨點跟著滑動（等於靜摩擦被突破）
   - 速度 > 0.5 m/s 或條件不成立 → 解除
   - 低速橫向阻尼（lowSpeedLateralDamping，不需要踩煞車也作用）：
     F = −c·v_y × (1 − smoothstep(1, 3, |v_x|))，c = 2·0.7·m_corner·(2π·6Hz)，限制在 μ_eff·Fz，與輪
胎 F_y 相加
     原因：|v_x| ≈ 0 時，鬆弛模型等效成一根阻尼很小的 6 Hz 側向彈簧，停車後會左右微晃

7. 這個階段先暫時用簡單驅動測試：後軸總扭力 T = throttle × 1500 N·m，經 open 差速器（stepDrivenAxle）分給兩
個後輪，
   煞車用 preset 的煞車扭力，轉向直接把前輪轉 steer × 30°。

Debug:
- 每輪顯示 κ、α（度）、F_x、F_y、gripUsage（顏色：綠 < 0.8、黃 0.8–1.0、紅 > 1.0）
- 在每輪上方畫一個摩擦圓小圖（圓 + 目前的力向量）
- 用 Tweakpane 調整全部輪胎參數；另外即時畫出 MF 曲線圖（縱向與側向），參數一改曲線就更新

單元測試（reference/tests 已有對應版本）：
- MF 曲線在 s = 1 處 = 1.0；s → ∞ 時趨近 sin(Cπ/2)
- 合成力在任意 κ、α 下 |F| ≤ μ_eff·Fz
- 平地靜止 + 未踩油門：5 秒後位移 < 1 cm
- 15% 坡道踩住煞車：5 秒內位移 < 2 cm
- 停車時給 0.2 m/s 側向速度：2 秒後 |v_y| < 0.001 m/s
- 鎖死煞車：ω 最終為 0 且不跨零震盪
- 不得產生 NaN（固定種子隨機輸入跑 10,000 步）
```

Phase 4：動力系統（引擎、變速箱、差速器、煞車、空力）

```
請先閱讀 AGENTS.md，完成 Phase 4：Engine、Gearbox、Differential、Brakes、Aero、Steering。移除 Phase 3
的暫時驅動。

1. Engine.ts
   - 扭力曲線使用 preset 表格，以 monotone cubic（或線性）插值
   - T_engine = throttle × T_curve(rpm) − (1 − throttle) × T_engineBrake(rpm)
   - 超過 limiter：切斷燃油（throttle 視為 0），並以 20 Hz 脈衝恢復，產生撞限轉的感覺
   - rpm 最低為 idle（怠速控制器：rpm < idle 時補扭力）
2. 離合器與自動起步
   - 名詞：wheelRpm = 驅動輪平均 ω × 目前檔比 × 終傳 換算的 rpm（也就是變速箱輸入端轉速）
   - 自動離合器（起步）：wheelRpm < launchRpm 時為半離合
     - 引擎轉速由離合器控制在 launchRpm = lerp(1200, 3000, throttle)，不得空轉衝高
     - 傳到變速箱的扭力 = 引擎在 launchRpm 的扭力 × throttle（離合器容量足夠時）
     - wheelRpm 追上 launchRpm → 完全結合，引擎 rpm = max(wheelRpm, idle)，剛性連結（在 launchRpm 結
合，轉速才不會突然掉而頓挫）
   - 引擎與傳動系的慣量要計入驅動輪的等效慣量：I_eff = I_w + I_engine × (gear × final)² / 驅動輪數
     （I_eff 就是 Wheel.ts 的 inertiaEff）
3. Gearbox.ts（自動模式為預設，也支援手動 shiftUp/shiftDown）
   - 所有換檔判斷一律用 wheelRpm，不用引擎轉速（起步半離合時引擎轉速和車速無關，用它會在起步時誤升檔）
   - upshiftRpm(throttle) = lerp(3500, 6500, throttle)；wheelRpm > upshiftRpm → 升檔
   - 降檔：wheelRpm < lerp(1800, 3500, throttle)（1 檔不再降）；或 kick-down：throttle > 0.9 且降檔後
的 wheelRpm < upshiftRpm(throttle) − 500
     （−500 的遲滯是必要的，否則 kick-down 門檻等於升檔點，會在兩檔之間來回跳）
   - 任何兩次換檔之間至少間隔 0.5 s（升檔、降檔都算）
   - 換檔期間 0.15 s 切斷扭力
   - 倒檔（Forza 行為，但避免坡道上踩住煞車就開始倒退）：
     - 進入：車速 < 1 km/h，且車子「停住之後」煞車有放開再踩下（煞車踩著一路停下來的話，不會進倒檔），brake >
0.5 持續 0.25 s
```

```
      - 倒檔中 brake 與 throttle 對調
      - 離開：倒檔中車速 < 1 km/h 且玩家按下原本的油門鍵（throttle > 0.5）→ 回到 1 檔
    - 轉彎中（側向 g > 0.6）延後升檔，避免彎中失去動力
4. Drivetrain.ts（差速器，以 reference/src/vehicle/Drivetrain.ts 為準）
    - 在每個車輪子步進計算一次，不是每個物理步長一次
    - open：左右扭力平分
    - lsd / locked：先算出讓兩輪「在這個子步進內剛好同速」所需的轉移扭力
      T_eq = |ω_l − ω_r| × I_eq / h，I_eq = I_l·I_r / (I_l + I_r)
      實際轉移 = min(容量, T_eq)，從快的一側移到慢的一側
      容量：lsd = preload + lockCoef × |T_in|；locked = ∞
    - 不可寫成「固定扭力 × sign(ω_l − ω_r)」（轉速接近時會每步翻轉而抖動），也不可用顯式強阻尼耦合（會發散）
    - AWD：中央差速器依前後分配比例分配扭力（中央也可用同樣的 min(容量, T_eq) 做法），再交給前後軸
5. Brakes.ts：brake 輸入 → 每輪扭力（依 preset 的前後最大值，前 2200 / 後 900）
    - 手煞車另外計算，只作用在後輪（2500 N·m），並且不經過 ABS（Phase 5），最後才與一般煞車扭力相加
6. Aero.ts：F_drag = 0.5ρ·CdA·v²（方向與速度相反，施加在 CoM）；
    下壓力 0.5ρ·ClA·v²，依前後比例施加在前後軸中點；滾動阻力 = Crr × Fz × r（作為 Wheel 的 resistTorque）
7. Steering.ts（只處理機構，輔助在 Phase 5）
    - 輸入 steer → 方向盤角，再乘上隨速度變化的上限：maxAngle(v) = max(6°, 33° / (1 + v / 15))，v 為 m/s
    - 輸出單一「中心輪角」δ（這就是 STM 用的 δ_driver）；Ackermann 不在這裡做，
      而是在 Phase 5 轉向輔助修正之後，最後一步才依 Ackermann 係數分成左右輪角（內側輪角較大）
    - 輸入平滑只在這裡做一次，依裝置使用 AGENTS.md 的數值（鍵盤、手把、方向盤各一組）；Phase 6 的輸入層只負責死區與曲線

HUD：速度 km/h、rpm 指針、檔位、油門/煞車條、手煞車指示。

單元測試與驗收：
- 差速器：左右抓地不同時 LSD 轉速差明顯小於 open，且轉速差不會來回翻轉（reference/tests 已有對應版本）
- 0→100 km/h 在 6.0–7.5 s（輔助全開、自動排檔）；reference/tests/longitudinal.test.ts 的簡化估算為 6.17 s，整車結果若差超過 1 s，先檢查離合器與換檔邏輯
- 起步全油門：車速 < 20 km/h 前不得升檔
- 極速 230–260 km/h
- 換檔時 rpm 連續，車身不會被猛拉
- 原地全油門時起步平順（Phase 5 開 TCS 之後再驗一次）
- 120 Hz 與 240 Hz 下 0→100 時間差 < 3%
```

Phase 5：輔助系統（讓手感變成 Forza 的關鍵）

```
請先閱讀 AGENTS.md，完成 Phase 5：assists/*。請以 reference/src/vehicle/assists/ 為準。
每個輔助系統都要能個別開關與調整強度，並提供三組 preset：
- forza_normal：ABS 開、TCS 開、STM 開、steeringAssist 開（強度 0.7）、countersteer 開、自動排檔
- sport：ABS 開、TCS 低（engageRatio 1.6、kp 0.15、ki 3）、STM 關、steeringAssist 0.3、countersteer 關、自動排檔
- sim：全部關閉、手動排檔

處理順序（每個物理步長）：
Steering（機構，中心輪角）→ steeringAssist → Ackermann 分成左右輪角 → TCS（引擎扭力倍率）→ STM（每輪附加煞車 + 油門倍率）→ ABS（每輪煞車壓力倍率）→ 加上手煞車扭力（不經 ABS）→ Wheel/Tire

1. steeringAssist.ts（最重要）
    - 最佳滑動角窗口：β_f = atan2(前軸側向速度, max(前軸前進速度, 1))，α_t = peakSlipAngle × 1.05
      玩家向右打（δ > 0）：δ_w = min(δ, max(β_f + α_t, 0))；向左打：δ_w = max(δ, min(β_f − α_t, 0))；不打：0
      也就是窗口只「限制幅度」，絕不主動把輪子轉到玩家沒打的那一側（反打交給下一項，避免疊加反打過頭）。
      再以 strength × smoothstep(5, 10, 速度) 與玩家輪角混合。
      效果：打滿方向盤時剛好在抓地極限，推頭時不會越打越推。
    - 自動反打：β_r = atan2(後軸側向速度, max(後軸前進速度, 1))；|β_r| > peakSlipAngle × 1.1（≈ 9°）時
      反打角 = 0.6 × (β_r − sign(β_r)·門檻)，限制在 ±15°，同樣乘上 fade。
      門檻不可低於後輪峰值滑動角，否則正常極限過彎時就會介入，感覺像推頭。
      玩家已經在反打（同號）時取較大的反打量，不跟玩家打架；異號時相加。
2. stm.ts（以 yaw rate 控制）
```

```
   - r_target = v·δ_driver / (L + K_us·v²)，K_us = 0.0025，限制在 ±1.0·μ·g / v（limitRatio 1.0；
0.85 會在極限過彎時誤觸）
   - δ_driver 是 Steering.ts 的機構輸出（轉向輔助「之前」）。不可用含自動反打的實際輪角，
     否則甩尾回正時 r_target 會指向另一側，STM 誤判成轉向不足而煞錯邊，把車甩到另一側。
   - 過度轉向（r 與 r_target 異號，或 |r| − |r_target| > 0.08 rad/s）：PI 控制煞「前外側輪」（上限 800
N·m），降低油門
   - 轉向不足（|r_target| − |r| > 0.08）：PI 控制輕煞「後內側輪」（上限 400 N·m），稍降油門
   - 輸出以一階低通 τ = 0.05 s 平滑；速度 < 15 km/h 時停用
   - 手煞車 > 0.1 時暫停（suspendWithHandbrake），放開後經同一個低通平滑恢復；否則手煞車甩尾會被立刻拉回
3. tcs.ts
   - 驅動輪最大 κ > peakSlipRatio × 1.15 → 啟動；以目標 κ = peakSlipRatio 的 PI 控制器（kp 0.25、ki 5）
降低引擎扭力倍率
   - 降扭立即生效，恢復速率上限 3 /s；手煞車拉著時停用（讓玩家可以甩尾）
   - 倒車時驅動方向相反，傳入前要把 κ 取反
   - 注意：kp 太大（例如 2）會在全砍與恢復之間循環，加速反而比沒開 TCS 還差
4. abs.ts
   - 輸入的 brakeCmd 是該輪的總煞車需求（玩家煞車 + STM 附加煞車），不包含手煞車；手煞車扭力在 ABS 之後才加上
   - 每輪 κ < −peakSlipRatio × 1.15 → 啟動；以目標 κ = −peakSlipRatio 的 PI 控制器（kp 1.5、ki 20）調
變煞車壓力
   - 不做 bang-bang 脈衝；另外輸出 15 Hz、±5% 的 hudPulse，只用於 HUD 與手把震動
   - 速度 < 5 km/h 時停用，讓車可以完全停下（之後由 Hold 接手）
5. 煞車時穩定性：全力直線煞車時，若出現 yaw，STM 優先修正
6. HUD：ABS / TCS / STM / ASSIST 四個指示燈，作用時亮起

驗收（forza_normal）：
- 100→0 煞車 36–42 m；煞車過程中打方向仍能轉向
- 半徑 50 m 定圓：穩態側向 0.95–1.05 g，油門加大時是平滑的推頭，不會突然甩尾
- 繞錐 60 km/h 不會失控；sim 模式下同樣的操作可以甩尾
- 手煞車進彎時可以甩尾，放開後 STM + 自動反打在 1 秒內把車救回
- 逐一開關每個輔助系統，行為差異要明顯且合理
```

Phase 6：輸入、鏡頭、回饋

```
請先閱讀 AGENTS.md，完成 Phase 6。

1. 輸入（core/input）
   - 鍵盤：W/S 或 ↑/↓ 油門/煞車，A/D 或 ←/→ 轉向，Space 手煞車，Q/E 換檔，C 切換鏡頭，R 重置，T 切換輔助
preset
     - 數位轉類比：油門 0→1 需 0.10 s、煞車 0.08 s；轉向的平滑在 Steering.ts（見 AGENTS.md），這裡不要重複
做
   - 手把（Gamepad API，標準 mapping）：左搖桿轉向、RT 油門、LT 煞車、A 手煞車、LB/RB 換檔
     - 死區 0.08（radial），反應曲線 pow(x, steerExponent)，steerExponent 預設 1.4，
       另外提供 steerLinearity 與 steerSensitivity 設定
     - 扳機死區 0.05
   - 方向盤：以 Gamepad API 讀取軸，用 calibration UI 讓玩家選軸與方向（力回饋不做）
   - 所有設定存在 localStorage
2. 追車鏡頭 ChaseCamera.ts（使用插值後的車身 transform，在渲染幀中更新）
   - 目標位置：車後 5.8 m、上方 1.7 m，看向車前 2 m、高 1.0 m 的點
   - 鏡頭以 critically damped spring 跟隨（位置 4 Hz、旋轉 6 Hz），yaw 以「速度方向」與「車頭方向」混合（甩
尾時能看到車身側面）
   - FOV：60° → 72°（隨 0–200 km/h 變化，平滑）
   - 加速時鏡頭微微後拉、煞車時前推（最大 0.3 m）
   - 顛簸時鏡頭依車身垂直加速度做輕微震動（可關閉）
   - 不得穿進地面（做簡單的地面高度檢查）
3. HoodCamera.ts：引擎蓋視角，加上依 pitch/roll 輕微晃動
4. 回饋
   - 手把震動（若支援 vibrationActuator）：gripUsage > 1 時弱震動、ABS 作用時跟著 hudPulse、碰撞時強震動
   - 地面胎痕：gripUsage > 1.1 時畫 decal 線條（ring buffer，最多 2000 段）
   - 胎煙：簡單粒子，gripUsage > 1.3 時產生
```

```
驗收：鍵盤與手把都能順暢駕駛；過彎時鏡頭不暈、不抖；以 30 fps 與 144 fps 渲染時操控感一致。
```

Phase 7：遙測、調參、Replay（調手感的工具）

```
請先閱讀 AGENTS.md，完成 Phase 7：調校工具。

1. Telemetry.ts：每個物理步長記錄一筆資料到 ring buffer（最近 20 秒）：
   速度、側向/縱向 g、yaw rate、r_target、車身滑動角 β、每輪的 Fz/κ/α/F_x/F_y/gripUsage/ω、
   steer 輸入與實際輪角、throttle/brake（玩家輸入與輔助修正後兩組）、rpm、檔位、各輔助作用量
2. Graphs.ts：可以選擇欄位的即時折線圖（canvas 2D，不要引入大型圖表套件），預設顯示：
   - 側向 g + yaw rate + r_target
   - 前後軸平均滑動角
   - 4 輪 gripUsage
   - g-g 圖（側向 vs 縱向加速度散佈圖）
3. TuningPanel.ts（Tweakpane）：分組列出 preset 所有參數；提供「重設為預設」「匯出 JSON」「匯入 JSON」「複製為 preset 檔」按鈕
4. Replay.ts：
   - 錄製：初始 VehicleState + 每步 VehicleInput
   - 播放：重新模擬並驗證最後的 state hash 一致（決定性測試）
   - 可以改參數後用同一段輸入重播，比較兩組遙測曲線（疊圖）
   - 匯出/匯入 .replay.json
5. 自動化測試腳本（vitest，headless 模式，不需渲染）：
   - acceleration_0_100、braking_100_0、skidpad_r50、slalom_60kmh、handbrake_turn、standstill_slope15
   - 每個測試輸出關鍵數值，並對照 AGENTS.md 的驗收目標
   - npm run handling-report 產生一份 markdown 報告（每個項目的數值與通過/失敗）

驗收：npm run handling-report 全部通過；replay 決定性測試通過。
```

三、試車時的「症狀 → 調整」對照表

開起來覺得哪裡不對時，把症狀加上 Phase 7 的遙測截圖貼給 Codex，請它依照這張表調整 preset：

症狀	優先調整
推頭，打方向盤車子不太轉	前輪 μ_lat 調高、前防傾桿調軟、peakSlipAngle 調小、K_us 調小
太容易甩尾	後防傾桿調軟、後輪 μ_lat 調高、C_lat 調低（滑動後保留更多抓地）、STM 門檻調低
方向盤反應太靈敏、太跳	σ_lat 鬆弛長度加大、轉向速率限制調慢、steerExponent 調高
方向盤反應太鈍、有延遲	σ_lat 調小、鏡頭旋轉 spring 頻率調高
車身像船一樣搖晃	阻尼調高（尤其 rebound）、彈簧與防傾桿調硬
車身太硬、過顛簸會彈跳	彈簧調軟、bump 阻尼調低
失控後很難救回來	countersteerGain 調高、STM 增益調高、C_lat 調低
正常過彎就覺得被輔助拉住	countersteerThreshold 調高、steeringAssist targetRatio 調高（例 1.15）
煞車點頭太多	前彈簧和前 bump 阻尼調高、CoM 高度調低
起步燒胎	TCS 門檻調低、1 檔齒比調低
TCS / ABS 作用時一頓一頓	kp 調低、ki 調低（先降 kp）
靜止時車子還在微抖或慢慢滑	低速橫向阻尼的阻尼比調高、V_EPS 與 V_RELAX_MIN 調大、Hold 頻率調高
高速太飄	ClA 調高、maxAngle(v) 隨速度下降得更快

四、跟 Codex 合作的建議

1. **一次只貼一個 Phase**。Phase 3 和 Phase 5 最容易出錯，完成後一定要自己開，並跑 handling-report。
2. **Phase 1–4 先別調手感**，物理正確（數值合理、不抖、測試通過）之後，Phase 5 才開始調「感覺」。
3. 提出調整時，請 Codex **只改 preset JSON 的數值，不改公式**，除非確定是公式的 bug。這樣比較好追蹤，也能用 replay 比較前後差異。
4. 想更貼近 Forza，可以錄一段自己開 Forza 的影片，用同樣的動作（定圓、繞錐、手煞車甩尾）在原型裡比對：進彎延遲、滑動後恢復的速度、車身側傾量。
5. 之後換成真實車輛模型時，只換 VehicleView 的模型，物理參數則依照車重、軸距、輪距修改 preset。

五、參考實作（TypeScript）

這份程式碼是 Phase 3、Phase 4 差速器與 Phase 5 的「標準答案」：

- **純邏輯**：不依賴 three.js 或 Rapier，可以直接在 Node 上測試，之後也能拿到多人伺服器上跑。
- 已通過 `tsc --strict`（含 `noUncheckedIndexedAccess`）與 **22 項 Vitest 單元測試**。
- 執行方式：`npm install`、`npm run typecheck`、`npm test`。
- 還沒有接上完整車輛（懸吊、動力系統、Rapier），那是 Codex 在 Phase 2 和 Phase 4 的工作。

符號約定總整理

量	正方向
車身 local	前 = −Z、右 = +X、上 = +Y
輪胎 v_x / v_y	沿滾動方向前進 / 接觸點往右滑
滑動角 α	往右滑為正；F_y 與 α 反號
滑動率 κ	驅動打滑為正、煞車鎖死為負
輪角 δ	向右為正
yawRateRight	車頭往右轉為正（= −ω_y）
輪序	0 = FL、1 = FR、2 = RL、3 = RR

四分之一車模擬驗證結果

單輪、車身只有縱向自由度、120 Hz、車輪子步進 4 次：

測試	結果
100→0 km/h 煞車，輪子鎖死	46.5 m
100→0 km/h 煞車，ABS	**38.5 m**（目標 36–42 m）
1500 N·m 起步 3 秒末速，無 TCS	24.95 m/s（輪子空轉）
1500 N·m 起步 3 秒末速，TCS	**30.83 m/s**，1 秒後最大 κ = 0.105（峰值 0.10）
15% 坡道踩煞車 5 秒，無駐車錨點	滑 3.40 cm
15% 坡道踩煞車 5 秒，有駐車錨點	**0.39 cm**
鎖死煞車 ω 跨零次數	0
停車側向晃動（第 2–3 秒最大 |v_y|），無低速橫向阻尼	0.011 m/s（超標）
停車側向晃動，有低速橫向阻尼	**≈ 0**
左右抓地不同（0.3 / 1.0）、1600 N·m，第 2–3 秒最大左右轉速差	open 150 rad/s、**LSD 0.2 rad/s（無抖動）**、locked 0.18 rad/s
STM：極限定圓 1.0 g、yaw 多 10%	limitRatio 1.0 不觸發；0.85 會誤觸
STM：拉手煞車時過度轉向	不介入；放開後 0.2 秒內恢復介入
隨機輸入 10,000 步	無 NaN

整車縱向估算

直線、準靜態前後重量轉移、預設 preset 的引擎扭力曲線、齒比、TCS、ABS、空力與滾阻：

項目	結果	驗收目標
0→100 km/h	**6.17 s**	6.0–7.5 s
極速	**253 km/h**	230–260 km/h
100→0 km/h（ABS）	**36.9 m**	36–42 m

src/vehicle/math.ts（數學工具）

```
export interface Vec3 {
  x: number;
  y: number;
  z: number;
}

export const v3 = (x = 0, y = 0, z = 0): Vec3 => ({ x, y, z });
export const add = (a: Vec3, b: Vec3): Vec3 => v3(a.x + b.x, a.y + b.y, a.z + b.z);
export const sub = (a: Vec3, b: Vec3): Vec3 => v3(a.x - b.x, a.y - b.y, a.z - b.z);
export const scale = (a: Vec3, s: number): Vec3 => v3(a.x * s, a.y * s, a.z * s);
export const dot = (a: Vec3, b: Vec3): number => a.x * b.x + a.y * b.y + a.z * b.z;
export const cross = (a: Vec3, b: Vec3): Vec3 =>
  v3(a.y * b.z - a.z * b.y, a.z * b.x - a.x * b.z, a.x * b.y - a.y * b.x);
export const length = (a: Vec3): number => Math.sqrt(dot(a, a));

/** 移除向量在法線方向的分量（投影到接觸平面） */
export const projectOnPlane = (a: Vec3, n: Vec3): Vec3 => sub(a, scale(n, dot(a, n)));

export const clamp = (x: number, lo: number, hi: number): number => Math.min(hi, Math.max(lo,
x));
export const lerp = (a: number, b: number, t: number): number => a + (b - a) * t;
export const smoothstep = (e0: number, e1: number, x: number): number => {
  const t = clamp((x - e0) / (e1 - e0), 0, 1);
  return t * t * (3 - 2 * t);
};
export const sign = (x: number): number => (x > 0 ? 1 : x < 0 ? -1 : 0);
export const DEG = Math.PI / 180;
export const GRAVITY = 9.81;
```

src/vehicle/Tire.ts（輪胎模型）

```
import { clamp } from './math';

/**
 * 輪胎模型：簡化 Pacejka（E = 0）+ 正規化合成滑動 + 負載敏感度 + 鬆弛長度。
 *
 * 符號約定（輪胎座標，位於接觸平面）：
 *   vx > 0：沿車輪滾動方向前進
 *   vy > 0：接觸點往車輪右側滑
 *   alpha = atan2(vy, |vx|)，往右滑為正
 *   fx > 0：往前推；fy 與 vy 反向（往右滑 → fy < 0，往左推回）
 */
export interface TireParams {
  peakSlipRatio: number; // 縱向峰值滑動率，例 0.10
  cLong: number; // 縱向形狀係數 C，須在 (1, 2)
  muLong: number;
  peakSlipAngleRad: number; // 側向峰值滑動角，例 8°
```

```
  cLat: number; // 側向形狀係數 C，須在 (1, 2)
  muLat: number;
  loadSensitivity: number; // 例 0.10
  nominalLoad: number; // N，例 3300
  relaxLenLat: number; // m，例 0.35
  relaxLenLong: number; // m，例 0.10
}

export const TIRE_CONSTS = {
  V_EPS: 0.5, // 滑動率/滑動角分母下限（m/s）
  V_RELAX_MIN: 1.0, // 鬆弛長度的最低等效速度（m/s）
  LOAD_FACTOR_MIN: 0.3,
  LOAD_FACTOR_MAX: 1.5,
} as const;

export interface TireState {
  slipRatio: number; // 經鬆弛後的 κ
  slipAngle: number; // 經鬆弛後的 α（rad）
}

export const createTireState = (): TireState => ({ slipRatio: 0, slipAngle: 0 });

export interface TireInput {
  vx: number;
  vy: number;
  omega: number; // 車輪角速度 rad/s，往前滾為正
  radius: number;
  fz: number; // 法向負載 N（≥ 0）
  surfaceGrip: number; // 路面倍率，乾 1.0 / 濕 0.7
  dt: number;
}

export interface TireOutput {
  fx: number;
  fy: number;
  slipRatio: number;
  slipAngle: number;
  /** 正規化合成滑動量 s：< 1 抓地區、= 1 峰值、> 1 滑動中 */
  gripUsage: number;
  muLongEff: number;
  muLatEff: number;
}

/**
 * 正規化 Magic Formula：s = 1 時為峰值 1.0，s → ∞ 時趨近 sin(Cπ/2)。
 * 等價於 sin(C·atan(B·x))，其中 B = tan(π/2C) / x_peak、s = x / x_peak。
 */
export function magicFormulaNormalized(s: number, c: number): number {
  const b = Math.tan(Math.PI / (2 * c));
  return Math.sin(c * Math.atan(b * s));
}

export function steadyStateSlip(vx: number, vy: number, omega: number, radius: number) {
  const wr = omega * radius;
  const denom = Math.max(Math.abs(vx), Math.abs(wr), TIRE_CONSTS.V_EPS);
  return {
    slipRatio: (wr - vx) / denom,
    slipAngle: Math.atan2(vy, Math.max(Math.abs(vx), TIRE_CONSTS.V_EPS)),
  };
}

/** 一階鬆弛的解析解係數，dt 再大也不會超過 1，所以不會發散 */
```

```
export function relaxFactor(vx: number, relaxLen: number, dt: number): number {
  return 1 - Math.exp((-(Math.abs(vx) + TIRE_CONSTS.V_RELAX_MIN) / relaxLen) * dt);
}

export function loadFactor(p: TireParams, fz: number): number {
  return clamp(
    1 - p.loadSensitivity * (fz / p.nominalLoad - 1),
    TIRE_CONSTS.LOAD_FACTOR_MIN,
    TIRE_CONSTS.LOAD_FACTOR_MAX,
  );
}

/** 只依 κ、α 計算合成力（無狀態），方便單元測試與畫曲線 */
export function combinedForce(
  p: TireParams,
  slipRatio: number,
  slipAngle: number,
  fz: number,
  surfaceGrip: number,
): Omit<TireOutput, 'slipRatio' | 'slipAngle'> {
  if (fz <= 0) return { fx: 0, fy: 0, gripUsage: 0, muLongEff: 0, muLatEff: 0 };

  const lf = loadFactor(p, fz);
  const muLongEff = p.muLong * surfaceGrip * lf;
  const muLatEff = p.muLat * surfaceGrip * lf;

  const sx = slipRatio / p.peakSlipRatio;
  const sy = Math.tan(slipAngle) / Math.tan(p.peakSlipAngleRad);
  const s = Math.hypot(sx, sy);
  if (s < 1e-9) return { fx: 0, fy: 0, gripUsage: 0, muLongEff, muLatEff };

  const fx = muLongEff * fz * magicFormulaNormalized(s, p.cLong) * (sx / s);
  const fy = -muLatEff * fz * magicFormulaNormalized(s, p.cLat) * (sy / s);
  return { fx, fy, gripUsage: s, muLongEff, muLatEff };
}

/**
 * 計算輪胎力。commit = true 時才把鬆弛後的滑動量寫回 state
 * （Wheel 會用 commit = false 做數值微分）。
 */
export function evaluateTire(
  p: TireParams,
  state: TireState,
  inp: TireInput,
  commit: boolean,
): TireOutput {
  const ss = steadyStateSlip(inp.vx, inp.vy, inp.omega, inp.radius);
  const kLong = relaxFactor(inp.vx, p.relaxLenLong, inp.dt);
  const kLat = relaxFactor(inp.vx, p.relaxLenLat, inp.dt);
  const slipRatio = state.slipRatio + (ss.slipRatio - state.slipRatio) * kLong;
  const slipAngle = state.slipAngle + (ss.slipAngle - state.slipAngle) * kLat;

  if (commit) {
    state.slipRatio = slipRatio;
    state.slipAngle = slipAngle;
  }
  return { ...combinedForce(p, slipRatio, slipAngle, inp.fz, inp.surfaceGrip), slipRatio,
slipAngle };
}
```

src/vehicle/Wheel.ts（車輪轉動（線性化隱式積分））

```
import { evaluateTire, type TireOutput, type TireParams, type TireState } from './Tire';

/**
 * 車輪轉動動力學（每個物理步長內做子步進）。
 *
 * I·dω/dt = T_drive − r·Fx(ω) − T_resist
 *
 * 低速時 ∂Fx/∂ω 非常大（V_EPS = 0.5 m/s 時約 1e5 N·s），顯式積分在 480 Hz 下也會發散。
 * 因此用「線性化隱式」：Fx(ω_new) ≈ Fx0 + K·(ω_new − ω0)，K = ∂Fx/∂ω（數值微分，≥ 0）
 *   ⇒ ω_new = ω0 + h·(T_drive − r·Fx0) / (I + h·r·K)
 * 煞車與滾阻屬於「不能讓 ω 變號」的阻力扭力：如果這一步會越過 0，就直接鎖在 0。
 *
 * 驅動輪之間有差速器耦合時，不能用 stepWheel 一次跑完一個輪子的所有子步進，
 * 必須每個子步進先算差速器分配（見 Drivetrain.ts 的 stepDrivenAxle），再對兩輪各呼叫一次 substepWheel。
 */
export interface WheelParams {
  radius: number;
  inertia: number; // 車輪自身慣量
}

export interface WheelState {
  omega: number;
  tire: TireState;
}

export interface WheelSubstepInput {
  vx: number; // 子步進期間車身接觸點速度固定不變
  vy: number;
  fz: number;
  surfaceGrip: number;
  driveTorque: number; // 有號，往前為正
  brakeTorque: number; // ≥ 0
  resistTorque: number; // ≥ 0，滾阻等
  inertiaEff: number; // 含引擎、傳動系的等效慣量
  h: number; // 子步進長度
}

export interface WheelSubstepOutput {
  tire: TireOutput;
  locked: boolean;
}

export function substepWheel(
  tp: TireParams,
  wp: WheelParams,
  st: WheelState,
  inp: WheelSubstepInput,
): WheelSubstepOutput {
  const h = inp.h;
  const r = wp.radius;
  const base = {
    vx: inp.vx,
    vy: inp.vy,
    omega: st.omega,
    radius: r,
    fz: inp.fz,
    surfaceGrip: inp.surfaceGrip,
    dt: h,
```

```
  };
  const f0 = evaluateTire(tp, st.tire, base, false);
  const eps = 1e-3 * Math.max(1, Math.abs(st.omega));
  const f1 = evaluateTire(tp, st.tire, { ...base, omega: st.omega + eps }, false);
  // 峰值後斜率為負，取 0 退化成顯式（該區斜率小，仍然穩定）
  const k = Math.max(0, (f1.fx - f0.fx) / eps);

  const denom = inp.inertiaEff + h * r * k;
  const free = st.omega + (h * (inp.driveTorque - r * f0.fx)) / denom;
  const resist = (h * (inp.brakeTorque + inp.resistTorque)) / denom;

  let locked = false;
  if (Math.abs(free) <= resist) {
    st.omega = 0;
    locked = inp.brakeTorque > 0;
  } else {
    st.omega = free - Math.sign(free) * resist;
  }

  const tire = evaluateTire(tp, st.tire, { ...base, omega: st.omega }, true);
  return { tire, locked };
}

export interface WheelStepInput extends Omit<WheelSubstepInput, 'h'> {
  dt: number; // 物理步長
  substeps: number; // 例 4
}

export interface WheelStepOutput {
  fx: number; // 子步進平均，作為衝量施加給車身
  fy: number;
  last: TireOutput;
  locked: boolean;
}

/** 沒有差速器耦合的輪子（非驅動輪，或測試用）可以直接用這個 */
export function stepWheel(
  tp: TireParams,
  wp: WheelParams,
  st: WheelState,
  inp: WheelStepInput,
): WheelStepOutput {
  const h = inp.dt / inp.substeps;
  let fxSum = 0;
  let fySum = 0;
  let out: WheelSubstepOutput | undefined;
  for (let i = 0; i < inp.substeps; i++) {
    out = substepWheel(tp, wp, st, { ...inp, h });
    fxSum += out.tire.fx;
    fySum += out.tire.fy;
  }
  return { fx: fxSum / inp.substeps, fy: fySum / inp.substeps, last: out!.tire, locked:
out!.locked };
}
```

src/vehicle/Hold.ts（低速駐車錨點與橫向阻尼）

```
import { add, dot, length, projectOnPlane, scale, sub, type Vec3 } from './math';

/**
```

```
 * 低速駐車力（錨點彈簧-阻尼）。
 *
 * 純阻尼無法抵抗持續外力（坡道重力分量），車會以等速慢慢滑。
 * 所以在靜止且踩著煞車時，記錄一個接觸點錨點，用彈簧把接觸點拉回去；
 * 需要的力超過 μ·Fz 時，力限制在 μ·Fz，錨點跟著滑動（等於靜摩擦被突破）。
 */
export interface HoldParams {
  engageSpeed: number; // 例 0.3 m/s
  releaseSpeed: number; // 例 0.5 m/s
  frequencyHz: number; // 例 3
  dampingRatio: number; // 例 1.0
}

export interface HoldState {
  active: boolean;
  anchor: Vec3;
}

/**
 * 低速橫向阻尼：|vx| 很小時，輪胎鬆弛模型等效成一根阻尼很小的側向彈簧（約 6 Hz），
 * 停車後會左右微晃。這個阻尼力在低速補上，速度變高後淡出。
 * 回傳沿 wheelRight 方向的力（N），請與輪胎 F_y 相加。
 */
export interface LateralDampingParams {
  fadeStart: number; // m/s，例 1
  fadeEnd: number; // m/s，例 3
  frequencyHz: number; // 例 6
  dampingRatio: number; // 例 0.7
}

export function lowSpeedLateralDamping(
  p: LateralDampingParams,
  vx: number,
  vy: number,
  fz: number,
  mu: number,
): number {
  if (fz <= 0) return 0;
  const speed = Math.abs(vx);
  if (speed >= p.fadeEnd) return 0;
  const t = speed <= p.fadeStart ? 0 : (speed - p.fadeStart) / (p.fadeEnd - p.fadeStart);
  const weight = 1 - t * t * (3 - 2 * t);
  const mCorner = fz / 9.81;
  const c = 2 * p.dampingRatio * mCorner * 2 * Math.PI * p.frequencyHz;
  const f = -c * vy * weight;
  const maxF = mu * fz;
  return Math.max(-maxF, Math.min(maxF, f));
}

export const createHoldState = (): HoldState => ({ active: false, anchor: { x: 0, y: 0, z: 0 }
});

export interface HoldInput {
  contactPoint: Vec3;
  contactVelocity: Vec3;
  normal: Vec3;
  fz: number;
  mu: number; // 有效摩擦係數（已含路面與負載）
  throttle: number;
  brake: number; // 已含手煞車（取 max）
}
```

```
/** 回傳要施加在接觸點的世界座標力 */
export function computeHoldForce(p: HoldParams, st: HoldState, inp: HoldInput): Vec3 {
  const zero = { x: 0, y: 0, z: 0 };
  const vPlanar = projectOnPlane(inp.contactVelocity, inp.normal);
  const speed = length(vPlanar);
  const wantsHold = inp.throttle < 0.05 && inp.brake > 0.05 && inp.fz > 0;

  if (!wantsHold || (st.active && speed > p.releaseSpeed)) {
    st.active = false;
    return zero;
  }
  if (!st.active) {
    if (speed > p.engageSpeed) return zero;
    st.active = true;
    st.anchor = { ...inp.contactPoint };
  }

  const mCorner = inp.fz / 9.81;
  const w = 2 * Math.PI * p.frequencyHz;
  const k = mCorner * w * w;
  const c = 2 * p.dampingRatio * mCorner * w;

  const offset = projectOnPlane(sub(inp.contactPoint, st.anchor), inp.normal);
  const force = add(scale(offset, -k), scale(vPlanar, -c));
  const maxForce = inp.mu * inp.fz;
  const mag = Math.sqrt(dot(force, force));

  if (mag > maxForce) {
    // 靜摩擦被突破：力限制在 μ·Fz，錨點跟著接觸點滑動，避免放開/抓住來回切換
    const ratio = maxForce / mag;
    st.anchor = sub(inp.contactPoint, scale(offset, ratio));
    return scale(force, ratio);
  }
  return force;
}
```

src/vehicle/Drivetrain.ts（差速器與驅動軸子步進）

```
import type { TireParams } from './Tire';
import {
  substepWheel,
  type WheelParams,
  type WheelState,
  type WheelSubstepInput,
  type WheelStepOutput,
} from './Wheel';

/**
 * 差速器。
 *
 * 不可寫成「固定扭力 × sign(ω_l − ω_r)」：兩輪轉速接近時正負號每一步翻轉，會高頻抖動。
 * 也不可用顯式強阻尼耦合 locked：阻尼一大就發散。
 *
 * 做法：先算出「讓兩輪在這個子步進內剛好同速」所需的轉移扭力 T_eq，
 * 實際轉移量 = min(容量, T_eq)。這樣差速器永遠不會讓轉速差反向（不會過衝），也就不會抖。
 *   T_eq = |Δω| · I_eq / h，I_eq = I_l·I_r / (I_l + I_r)
 *   容量：open = 0、lsd = preload + lockCoef·|T_in|、locked = ∞
 * 轉移方向：從轉得快的一側移到慢的一側。
 */
export type DiffType = 'open' | 'lsd' | 'locked';
```

```
export interface DiffParams {
  type: DiffType;
  preload: number; // N·m, 例 50
  lockCoef: number; // 例 0.25
}

/** 回傳 [左輪驅動扭力, 右輪驅動扭力] */
export function splitDiffTorque(
  p: DiffParams,
  torqueIn: number,
  omegaL: number,
  omegaR: number,
  inertiaL: number,
  inertiaR: number,
  h: number,
): [number, number] {
  const half = torqueIn / 2;
  if (p.type === 'open') return [half, half];

  const dOmega = omegaL - omegaR;
  const iEq = (inertiaL * inertiaR) / (inertiaL + inertiaR);
  const tEq = (Math.abs(dOmega) * iEq) / h;
  const capacity = p.type === 'locked' ? Infinity : p.preload + p.lockCoef * Math.abs(torqueIn);
  const transfer = Math.min(capacity, tEq) * Math.sign(dOmega);
  return [half - transfer, half + transfer];
}

export interface AxleWheelInput extends Omit<WheelSubstepInput, 'driveTorque' | 'h'> {}

/**
 * 驅動軸: 每個子步進先分配差速器扭力，再推進兩輪。
 * AWD 時前後軸各呼叫一次（中央差速器先把引擎扭力依比例分給前後軸）。
 */
export function stepDrivenAxle(
  tp: TireParams,
  wp: WheelParams,
  states: [WheelState, WheelState],
  inputs: [AxleWheelInput, AxleWheelInput],
  diff: DiffParams,
  axleTorque: number,
  dt: number,
  substeps: number,
): [WheelStepOutput, WheelStepOutput] {
  const h = dt / substeps;
  const sums = [
    { fx: 0, fy: 0 },
    { fx: 0, fy: 0 },
  ];
  const outs: [WheelStepOutput | undefined, WheelStepOutput | undefined] = [undefined,
undefined];

  for (let i = 0; i < substeps; i++) {
    const [tL, tR] = splitDiffTorque(
      diff,
      axleTorque,
      states[0].omega,
      states[1].omega,
      inputs[0].inertiaEff,
      inputs[1].inertiaEff,
      h,
    );
```

```
    const torques = [tL, tR];
    for (let w = 0; w < 2; w++) {
      const o = substepWheel(tp, wp, states[w]!, { ...inputs[w]!, driveTorque: torques[w]!, h });
      sums[w]!.fx += o.tire.fx;
      sums[w]!.fy += o.tire.fy;
      outs[w] = { fx: 0, fy: 0, last: o.tire, locked: o.locked };
    }
  }

  return [0, 1].map((w) => ({
    ...outs[w]!,
    fx: sums[w]!.fx / substeps,
    fy: sums[w]!.fy / substeps,
  })) as [WheelStepOutput, WheelStepOutput];
}
```

src/vehicle/assists/PI.ts（PI 控制器與低通）

```
import { clamp } from '../math';

/** 帶 anti-windup 的 PI 控制器，積分項限制在 [0, iMax] */
export interface PIState {
  integ: number;
}

export function updatePI(
  st: PIState,
  error: number,
  kp: number,
  ki: number,
  iMax: number,
  dt: number,
): number {
  st.integ = clamp(st.integ + error * dt, 0, iMax);
  return kp * error + ki * st.integ;
}

/** 一階低通，tau 為時間常數 */
export function lowPass(prev: number, target: number, tau: number, dt: number): number {
  if (tau <= 0) return target;
  return prev + (target - prev) * (1 - Math.exp(-dt / tau));
}
```

src/vehicle/assists/abs.ts（ABS）

```
import { clamp } from '../math';
import { updatePI, type PIState } from './PI';

/**
 * ABS：以「目標滑動率」做 PI 調變，不做生硬的 bang-bang 脈衝。
 * 回傳 0..1 的煞車壓力倍率，每輪各自一個 state。
 */
export interface AbsParams {
  enabled: boolean;
  peakSlipRatio: number;
  engageRatio: number; // κ < −peak × 1.15 才啟動
  targetRatio: number; // 目標 κ = −peak × 1.0
```

```
  releaseRatio: number; // κ > −peak × 0.5 且積分歸零才解除
  kp: number; // 例 1.5
  ki: number; // 例 20
  minPressure: number; // 例 0.05
  minSpeed: number; // m/s，例 5 km/h = 1.39
  hudPulseHz: number; // 只給 HUD/震動用，例 15
}

export interface AbsWheelState extends PIState {
  active: boolean;
  pressure: number;
  phase: number;
}

export const createAbsState = (): AbsWheelState => ({ active: false, pressure: 1, phase: 0,
integ: 0 });

export interface AbsOutput {
  pressureScale: number;
  active: boolean;
  /** 15 Hz ±5% 的回饋訊號，不影響物理 */
  hudPulse: number;
}

/** brakeCmd: 該輪的總煞車需求（玩家煞車 + STM 附加煞車），正規化到 0..1 */
export function updateAbs(
  p: AbsParams,
  st: AbsWheelState,
  slipRatio: number,
  forwardSpeed: number,
  brakeCmd: number,
  dt: number,
): AbsOutput {
  if (!p.enabled || Math.abs(forwardSpeed) < p.minSpeed || brakeCmd < 0.01) {
    st.active = false;
    st.integ = 0;
    st.pressure = 1;
    return { pressureScale: 1, active: false, hudPulse: 0 };
  }

  // 倒車時 κ 的意義相反，統一換成「煞車方向的滑動」
  const k = forwardSpeed >= 0 ? slipRatio : -slipRatio;
  if (!st.active && k < -p.peakSlipRatio * p.engageRatio) st.active = true;

  if (st.active) {
    const target = -p.peakSlipRatio * p.targetRatio;
    const e = (target - k) / p.peakSlipRatio; // > 0: 滑太多
    const u = updatePI(st, e, p.kp, p.ki, 1 / p.ki, dt);
    st.pressure = clamp(1 - u, p.minPressure, 1);
    if (st.integ === 0 && k > -p.peakSlipRatio * p.releaseRatio) st.active = false;
  } else {
    st.pressure = 1;
  }

  st.phase = (st.phase + p.hudPulseHz * dt) % 1;
  const hudPulse = st.active ? 0.05 * Math.sin(st.phase * 2 * Math.PI) : 0;
  return { pressureScale: st.pressure, active: st.active, hudPulse };
}
```

src/vehicle/assists/tcs.ts（TCS）

```
import { clamp } from '../math';
import { updatePI, type PIState } from './PI';

/**
 * TCS：驅動輪 κ 超過門檻 → 降低引擎扭力倍率。
 * 降扭立即生效，恢復則以 recoverRate 慢慢回升，避免油門忽大忽小。
 */
export interface TcsParams {
  enabled: boolean;
  peakSlipRatio: number;
  engageRatio: number; // 例 1.15
  targetRatio: number; // 例 1.0
  kp: number; // 例 0.25（太大會在全砍/恢復之間循環，平均扭力反而變少）
  ki: number; // 例 5
  recoverRate: number; // 每秒恢復量，例 3
  disableWithHandbrake: boolean;
}

export interface TcsState extends PIState {
  scale: number;
  active: boolean;
}

export const createTcsState = (): TcsState => ({ scale: 1, active: false, integ: 0 });

/** maxDrivenSlip：驅動輪中最大的 κ（倒車時請先取反） */
export function updateTcs(
  p: TcsParams,
  st: TcsState,
  maxDrivenSlip: number,
  handbrake: number,
  dt: number,
): number {
  if (!p.enabled || (p.disableWithHandbrake && handbrake > 0.1)) {
    st.integ = 0;
    st.active = false;
    st.scale = Math.min(1, st.scale + p.recoverRate * dt);
    return st.scale;
  }

  const e = (maxDrivenSlip - p.peakSlipRatio * p.targetRatio) / p.peakSlipRatio;
  if (!st.active && maxDrivenSlip > p.peakSlipRatio * p.engageRatio) st.active = true;

  let cmd = 1;
  if (st.active) {
    const u = updatePI(st, e, p.kp, p.ki, 1 / p.ki, dt);
    cmd = clamp(1 - Math.max(0, u), 0, 1);
    if (st.integ === 0 && e < 0) st.active = false;
  }

  st.scale = cmd < st.scale ? cmd : Math.min(cmd, st.scale + p.recoverRate * dt);
  return st.scale;
}
```

src/vehicle/assists/stm.ts（STM 穩定控制）

```
import { clamp, GRAVITY, sign } from '../math';
import { lowPass, updatePI, type PIState } from './PI';

/**
 * STM（穩定控制），以 yaw rate 為控制量。
 *
 * 符號：steerAngle > 0 = 向右；yawRateRight > 0 = 車頭往右轉。
 * （Y-up 右手座標下 ω_y > 0 是往左轉，所以 yawRateRight = −ω_y）
 *
 * 輪序：0 = FL, 1 = FR, 2 = RL, 3 = RR
 */
export interface StmParams {
  enabled: boolean;
  minSpeed: number; // m/s，例 15 km/h = 4.17
  wheelbase: number;
  kUs: number; // 不足轉向梯度，例 0.0025
  muEstimate: number; // 例 1.05
  limitRatio: number; // 例 1.0；0.85 時極限過彎的正常 yaw 波動（約 10%）就會觸發
  deadband: number; // rad/s，例 0.08
  kp: number; // N·m / (rad/s)，例 2500
  ki: number; // 例 4000
  maxFrontBrake: number; // 例 800
  maxRearBrake: number; // 例 400
  throttleCutGain: number; // 例 2.0
  minThrottleScale: number; // 例 0.2
  filterTau: number; // s，例 0.05
  /** 拉手煞車時暫停，否則手煞車甩尾會被 STM 立刻拉回；放開後經 filterTau 平滑恢復 */
  suspendWithHandbrake: boolean;
}

export interface StmState {
  over: PIState;
  under: PIState;
  brake: [number, number, number, number];
  throttleScale: number;
  mode: 'none' | 'oversteer' | 'understeer';
}

export const createStmState = (): StmState => ({
  over: { integ: 0 },
  under: { integ: 0 },
  brake: [0, 0, 0, 0],
  throttleScale: 1,
  mode: 'none',
});

export interface StmInput {
  forwardSpeed: number;
  /**
   * 駕駛意圖的輪角：Steering.ts 的機構輸出，也就是 steeringAssist「之前」的值。
   * 不可用含自動反打的實際輪角，否則甩尾回正時，反打角會讓 r_target 指向另一側，
   * STM 誤判成轉向不足，煞錯邊的後輪，把車甩到另一側。
   */
  driverSteerAngle: number;
  yawRateRight: number;
  handbrake: number; // 0..1
  dt: number;
}
```

```
export interface StmOutput {
  brakeTorque: [number, number, number, number];
  throttleScale: number;
  yawRateTarget: number;
  mode: StmState['mode'];
}

export function yawRateTarget(p: StmParams, v: number, steerAngle: number): number {
  const r = (v * steerAngle) / (p.wheelbase + p.kUs * v * v);
  const limit = (p.limitRatio * p.muEstimate * GRAVITY) / Math.max(v, 1);
  return clamp(r, -limit, limit);
}

export function updateStm(p: StmParams, st: StmState, inp: StmInput): StmOutput {
  const v = inp.forwardSpeed;
  const target = [0, 0, 0, 0] as [number, number, number, number];
  let throttleTarget = 1;
  let rT = 0;
  st.mode = 'none';

  const suspended = p.suspendWithHandbrake && inp.handbrake > 0.1;
  if (p.enabled && !suspended && v >= p.minSpeed) {
    rT = yawRateTarget(p, v, inp.driverSteerAngle);
    const r = inp.yawRateRight;
    const opposite = sign(r) !== 0 && sign(rT) !== 0 && sign(r) !== sign(rT);
    const overErr = opposite ? Math.abs(r) + Math.abs(rT) : Math.abs(r) - Math.abs(rT);
    const underErr = opposite ? 0 : Math.abs(rT) - Math.abs(r);

    if (Math.abs(r) > p.deadband && overErr > p.deadband) {
      // 過度轉向：煞「前外側輪」產生反向 yaw。往右轉（r > 0）時外側是左前
      st.mode = 'oversteer';
      st.under.integ = 0;
      const e = overErr - p.deadband;
      const tq = clamp(updatePI(st.over, e, p.kp, p.ki, p.maxFrontBrake / p.ki, inp.dt), 0,
p.maxFrontBrake);
      target[r > 0 ? 0 : 1] = tq;
      throttleTarget = 1 - p.throttleCutGain * e;
    } else if (underErr > p.deadband) {
      // 轉向不足：輕煞「後內側輪」幫助車頭轉進去。想往右轉（rT > 0）時內側是右後
      st.mode = 'understeer';
      st.over.integ = 0;
      const e = underErr - p.deadband;
      const tq = clamp(updatePI(st.under, e, p.kp, p.ki, p.maxRearBrake / p.ki, inp.dt), 0,
p.maxRearBrake);
      target[rT > 0 ? 3 : 2] = tq;
      throttleTarget = 1 - 0.5 * p.throttleCutGain * e;
    } else {
      st.over.integ = 0;
      st.under.integ = 0;
    }
  } else {
    st.over.integ = 0;
    st.under.integ = 0;
  }

  for (let i = 0; i < 4; i++) st.brake[i] = lowPass(st.brake[i]!, target[i]!, p.filterTau,
inp.dt);
  st.throttleScale = lowPass(
    st.throttleScale,
    clamp(throttleTarget, p.minThrottleScale, 1),
    p.filterTau,
```

```
    inp.dt,
  );

  return {
    brakeTorque: [...st.brake] as [number, number, number, number],
    throttleScale: st.throttleScale,
    yawRateTarget: rT,
    mode: st.mode,
  };
}
```

src/vehicle/assists/steeringAssist.ts（轉向輔助與自動反打）

```
import { clamp, lerp, sign, smoothstep } from '../math';

/**
 * 轉向輔助：最佳滑動角窗口 + 自動反打。
 *
 * 符號：輪角 > 0 = 向右。軸速度用車身 local 分解：
 *   lateral > 0 = 往右移動，forward > 0 = 往前。
 *
 * 前輪滑動角 α ≈ β_f − δ，其中 β_f = atan2(lateral_f, forward)。
 * 讓 |α| ≤ α_target ⇒ δ ∈ [β_f − α_target, β_f + α_target]。
 * 玩家打滿方向盤時，輪角被限制在這個窗口內，所以「打到底剛好在抓地極限」，
 * 推頭時不會越打越推。窗口只限制幅度，不會讓輪子越過中心轉到玩家沒打的那一側。
 */
export interface SteeringAssistParams {
  enabled: boolean;
  strength: number; // 0..1，例 0.7
  peakSlipAngleRad: number;
  targetRatio: number; // 例 1.05
  fadeStart: number; // m/s，例 5
  fadeEnd: number; // m/s，例 10
  countersteerEnabled: boolean;
  countersteerGain: number; // 例 0.6
  countersteerThresholdRad: number; // 例 peakSlipAngle × 1.1 ≈ 9°；低於峰值時正常極限過彎就會介入，感
覺像推頭
  countersteerMaxRad: number; // 例 15°
  maxWheelAngleRad: number; // 機構極限，例 33°
}

export interface AxleVelocity {
  lateral: number;
  forward: number;
}

export interface SteeringAssistInput {
  playerAngle: number; // Steering.ts 輸出的輪角（已含速度敏感上限）
  frontAxle: AxleVelocity;
  rearAxle: AxleVelocity;
}

export interface SteeringAssistOutput {
  angle: number;
  limited: boolean;
  countersteer: number;
  rearSlipAngle: number;
}

export function updateSteeringAssist(
```

```
  p: SteeringAssistParams,
  inp: SteeringAssistInput,
): SteeringAssistOutput {
  const fwd = inp.frontAxle.forward;
  const passthrough = {
    angle: inp.playerAngle,
    limited: false,
    countersteer: 0,
    rearSlipAngle: 0,
  };
  if (!p.enabled || fwd < 1) return passthrough;

  const fade = smoothstep(p.fadeStart, p.fadeEnd, fwd) * p.strength;

  const betaF = Math.atan2(inp.frontAxle.lateral, Math.max(fwd, 1));
  const alphaT = p.peakSlipAngleRad * p.targetRatio;
  // 窗口只「限制」玩家往某一側打的量，不會主動把輪子轉到另一側；反打交給下面的 countersteer，
  // 否則兩者疊加會反打過頭
  const player = inp.playerAngle;
  const windowed =
    player > 0
      ? Math.min(player, Math.max(betaF + alphaT, 0))
      : player < 0
        ? Math.max(player, Math.min(betaF - alphaT, 0))
        : 0;
  let angle = lerp(inp.playerAngle, windowed, fade);
  const limited = Math.abs(windowed - inp.playerAngle) > 1e-4 && fade > 0;

  // 後軸往左滑（β_r < 0，例如右彎甩尾）⇒ 反打方向是向左（負）
  const betaR = Math.atan2(inp.rearAxle.lateral, Math.max(inp.rearAxle.forward, 1));
  let cs = 0;
  if (p.countersteerEnabled && Math.abs(betaR) > p.countersteerThresholdRad) {
    cs = clamp(
      p.countersteerGain * (betaR - sign(betaR) * p.countersteerThresholdRad),
      -p.countersteerMaxRad,
      p.countersteerMaxRad,
    ) * fade;
    if (sign(angle) === sign(cs) || angle === 0) {
      // 玩家已經在反打：取較大的反打量，不跟玩家打架
      angle = sign(cs) * Math.max(Math.abs(angle), Math.abs(cs));
    } else {
      angle += cs;
    }
  }

  return {
    angle: clamp(angle, -p.maxWheelAngleRad, p.maxWheelAngleRad),
    limited,
    countersteer: cs,
    rearSlipAngle: betaR,
  };
}
```

tests/handling.test.ts（單元測試（Vitest））

```
import { describe, expect, it } from 'vitest';
import { DEG, GRAVITY } from '../src/vehicle/math';
import { combinedForce, createTireState, magicFormulaNormalized, type TireParams } from
'../src/vehicle/Tire';
import { stepWheel, type WheelState } from '../src/vehicle/Wheel';
```

```
import { stepDrivenAxle, type DiffParams } from '../src/vehicle/Drivetrain';
import { computeHoldForce, createHoldState, lowSpeedLateralDamping, type HoldParams } from
'../src/vehicle/Hold';
import { evaluateTire } from '../src/vehicle/Tire';
import { createAbsState, updateAbs, type AbsParams } from '../src/vehicle/assists/abs';
import { createTcsState, updateTcs, type TcsParams } from '../src/vehicle/assists/tcs';
import { createStmState, updateStm, type StmParams } from '../src/vehicle/assists/stm';
import { updateSteeringAssist, type SteeringAssistParams } from
'../src/vehicle/assists/steeringAssist';

const tire: TireParams = {
  peakSlipRatio: 0.1,
  cLong: 1.5,
  muLong: 1.1,
  peakSlipAngleRad: 8 * DEG,
  cLat: 1.3,
  muLat: 1.05,
  loadSensitivity: 0.1,
  nominalLoad: 3300,
  relaxLenLat: 0.35,
  relaxLenLong: 0.1,
};
const wheel = { radius: 0.33, inertia: 1.0 };
const absP: AbsParams = {
  enabled: true,
  peakSlipRatio: 0.1,
  engageRatio: 1.15,
  targetRatio: 1.0,
  releaseRatio: 0.5,
  kp: 1.5,
  ki: 20,
  minPressure: 0.05,
  minSpeed: 1.39,
  hudPulseHz: 15,
};
const tcsP: TcsParams = {
  enabled: true,
  peakSlipRatio: 0.1,
  engageRatio: 1.15,
  targetRatio: 1.0,
  kp: 0.25,
  ki: 5,
  recoverRate: 3,
  disableWithHandbrake: true,
};
const holdP: HoldParams = { engageSpeed: 0.3, releaseSpeed: 0.5, frequencyHz: 3, dampingRatio: 1
};

const DT = 1 / 120;
const MASS = 337.5;
const FZ = MASS * GRAVITY;

interface SimOpts {
  v0: number;
  seconds: number;
  drive?: number;
  brake?: number;
  abs?: boolean;
  tcs?: boolean;
  slopeForce?: number;
  hold?: boolean;
}
```

```
/** 單輪縱向模擬：車身只有 x 方向自由度 */
function quarterCar(o: SimOpts) {
  const st: WheelState = { omega: o.v0 / wheel.radius, tire: createTireState() };
  const absS = createAbsState();
  const tcsS = createTcsState();
  const holdS = createHoldState();
  let v = o.v0;
  let x = 0;
  let maxSlip = 0;
  let omegaSignFlips = 0;
  let nan = false;
  let stopDist: number | null = null;
  let tcsSlipAfter1s = 0;

  for (let i = 0; i < o.seconds / DT; i++) {
    let brake = o.brake ?? 0;
    let drive = o.drive ?? 0;
    if (o.abs) brake *= updateAbs(absP, absS, st.tire.slipRatio, v, brake > 0 ? 1 : 0,
DT).pressureScale;
    if (o.tcs) drive *= updateTcs(tcsP, tcsS, st.tire.slipRatio, 0, DT);

    const prevOmega = st.omega;
    const out = stepWheel(tire, wheel, st, {
      vx: v,
      vy: 0,
      fz: FZ,
      surfaceGrip: 1,
      driveTorque: drive,
      brakeTorque: brake,
      resistTorque: 0.012 * FZ * wheel.radius,
      inertiaEff: wheel.inertia,
      dt: DT,
      substeps: 4,
    });
    if (prevOmega * st.omega < 0) omegaSignFlips++;

    let fHold = 0;
    if (o.hold) {
      fHold = computeHoldForce(holdP, holdS, {
        contactPoint: { x, y: 0, z: 0 },
        contactVelocity: { x: v, y: 0, z: 0 },
        normal: { x: 0, y: 1, z: 0 },
        fz: FZ,
        mu: tire.muLong,
        throttle: 0,
        brake: (o.brake ?? 0) > 0 ? 1 : 0,
      }).x;
    }

    v += ((out.fx + fHold + (o.slopeForce ?? 0)) / MASS) * DT;
    x += v * DT;
    maxSlip = Math.max(maxSlip, out.last.slipRatio);
    if (i * DT > 1 && o.tcs) tcsSlipAfter1s = Math.max(tcsSlipAfter1s, out.last.slipRatio);
    if (!Number.isFinite(v) || !Number.isFinite(st.omega)) nan = true;
    if (stopDist === null && o.v0 > 0 && v <= 0.05) stopDist = x;
  }
  return { v, x, omega: st.omega, maxSlip, omegaSignFlips, nan, stopDist, tcsSlipAfter1s };
}

describe('Magic Formula', () => {
  it('s = 1 時為峰值 1.0', () => {
```

```
    expect(magicFormulaNormalized(1, 1.5)).toBeCloseTo(1, 6);
    expect(magicFormulaNormalized(1, 1.3)).toBeCloseTo(1, 6);
  });
  it('s → ∞ 時趨近 sin(Cπ/2)', () => {
    expect(magicFormulaNormalized(1e6, 1.3)).toBeCloseTo(Math.sin((1.3 * Math.PI) / 2), 3);
  });
  it('合成力永遠不超過 μ_eff·Fz', () => {
    for (let k = -1; k <= 1; k += 0.05)
      for (let a = -60; a <= 60; a += 2) {
        const f = combinedForce(tire, k, a * DEG, FZ, 1);
        const mu = Math.max(f.muLongEff, f.muLatEff);
        expect(Math.hypot(f.fx, f.fy)).toBeLessThanOrEqual(mu * FZ + 1e-6);
      }
  });
  it('側向力方向與滑動方向相反', () => {
    expect(combinedForce(tire, 0, 5 * DEG, FZ, 1).fy).toBeLessThan(0);
    expect(combinedForce(tire, 0, -5 * DEG, FZ, 1).fy).toBeGreaterThan(0);
  });
});

describe('車輪積分（四分之一車）', () => {
  it('全力煞車鎖死：ω 停在 0、不跨零震盪、能停下', () => {
    const r = quarterCar({ v0: 27.8, seconds: 6, brake: 2200, hold: true });
    expect(r.nan).toBe(false);
    expect(r.omegaSignFlips).toBe(0);
    expect(Math.abs(r.v)).toBeLessThan(0.05);
  });
  it('ABS 煞車距離比鎖死短', () => {
    const locked = quarterCar({ v0: 27.8, seconds: 6, brake: 2200 });
    const abs = quarterCar({ v0: 27.8, seconds: 6, brake: 2200, abs: true });
    console.log(`100→0 鎖死 ${locked.stopDist?.toFixed(1)} m，ABS ${abs.stopDist?.toFixed(1)} m`);
    expect(abs.stopDist!).toBeLessThan(locked.stopDist!);
    expect(abs.stopDist!).toBeGreaterThan(30);
    expect(abs.stopDist!).toBeLessThan(42);
  });
  it('低速起步大扭力不會 NaN 或發散', () => {
    const r = quarterCar({ v0: 0, seconds: 3, drive: 1500 });
    expect(r.nan).toBe(false);
    expect(r.v).toBeGreaterThan(5);
  });
  it('TCS 把起步滑動率壓在峰值附近，而且加速較好', () => {
    const raw = quarterCar({ v0: 0, seconds: 3, drive: 1500 });
    const tcs = quarterCar({ v0: 0, seconds: 3, drive: 1500, tcs: true });
    console.log(`3 秒末速 無TCS ${raw.v.toFixed(2)} m/s，TCS ${tcs.v.toFixed(2)} m/s；1 秒後最大 κ
${tcs.tcsSlipAfter1s.toFixed(3)}`);
    expect(tcs.tcsSlipAfter1s).toBeLessThan(0.2);
    expect(tcs.v).toBeGreaterThan(raw.v);
  });
  it('平地靜止 5 秒位移 < 1 cm', () => {
    const r = quarterCar({ v0: 0, seconds: 5 });
    expect(Math.abs(r.x)).toBeLessThan(0.01);
  });
  it('15% 坡道踩煞車 5 秒位移 < 2 cm（駐車錨點）', () => {
    const slope = -MASS * GRAVITY * Math.sin(Math.atan(0.15));
    const noHold = quarterCar({ v0: 0, seconds: 5, brake: 2200, slopeForce: slope });
    const hold = quarterCar({ v0: 0, seconds: 5, brake: 2200, slopeForce: slope, hold: true });
    console.log(`坡道位移 無錨點 ${(noHold.x * 100).toFixed(2)} cm，有錨點 ${(hold.x *
100).toFixed(2)} cm`);
    expect(Math.abs(hold.x)).toBeLessThan(0.02);
  });
  it('停車後的側向晃動：加上低速橫向阻尼後 2 秒內收斂', () => {
    for (const damp of [false, true]) {
```

```
      const st = createTireState();
      let vy = 0.2;
      let maxLate = 0;
      for (let i = 0; i < 360; i++) {
        const o = evaluateTire(tire, st, { vx: 0, vy, omega: 0, radius: 0.33, fz: FZ,
surfaceGrip: 1, dt: DT }, true);
        const fd = damp
          ? lowSpeedLateralDamping({ fadeStart: 1, fadeEnd: 3, frequencyHz: 6, dampingRatio: 0.7
}, 0, vy, FZ, 1.05)
          : 0;
        vy += ((o.fy + fd) / MASS) * DT;
        if (i > 240) maxLate = Math.max(maxLate, Math.abs(vy));
      }
      console.log(`停車側向晃動 第 2–3 秒最大 |vy|: ${damp ? '有阻尼' : '無阻尼'}
${maxLate.toExponential(2)} m/s`);
      if (damp) expect(maxLate).toBeLessThan(0.001);
    }
  });
  it('120 Hz 隨機輸入 10,000 步不會 NaN', () => {
    let seed = 1;
    const rnd = () => ((seed = (seed * 16807) % 2147483647) / 2147483647);
    const st: WheelState = { omega: 0, tire: createTireState() };
    let v = 0;
    for (let i = 0; i < 10000; i++) {
      const out = stepWheel(tire, wheel, st, {
        vx: v,
        vy: (rnd() - 0.5) * 10,
        fz: FZ * rnd() * 2,
        surfaceGrip: 0.5 + rnd() * 0.5,
        driveTorque: (rnd() - 0.3) * 3000,
        brakeTorque: rnd() < 0.3 ? rnd() * 3000 : 0,
        resistTorque: 10,
        inertiaEff: 1,
        dt: DT,
        substeps: 4,
      });
      v += (out.fx / MASS) * DT;
      expect(Number.isFinite(out.fx) && Number.isFinite(out.fy) &&
Number.isFinite(st.omega)).toBe(true);
    }
  });
});

describe('差速器（左右抓地不同: 左 0.3 / 右 1.0）', () => {
  function splitMu(diff: DiffParams, seconds = 3) {
    const states: [WheelState, WheelState] = [
      { omega: 0, tire: createTireState() },
      { omega: 0, tire: createTireState() },
    ];
    let v = 0;
    let flips = 0;
    let prevD = 0;
    let maxLateDelta = 0;
    for (let i = 0; i < seconds / DT; i++) {
      const common = { vy: 0, fz: FZ, brakeTorque: 0, resistTorque: 10, inertiaEff: 8 };
      const [l, r] = stepDrivenAxle(
        tire,
        wheel,
        states,
        [
          { ...common, vx: v, surfaceGrip: 0.3 },
          { ...common, vx: v, surfaceGrip: 1.0 },
```

```
        ],
        diff,
        1600,
        DT,
        4,
      );
      v += ((l.fx + r.fx) / (2 * MASS)) * DT;
      const d = states[0].omega - states[1].omega;
      if (i > 1 / DT && Math.abs(d) > 1e-6 && Math.abs(prevD) > 1e-6 && d * prevD < 0) flips++;
      if (i > 2 / DT) maxLateDelta = Math.max(maxLateDelta, Math.abs(d));
      prevD = d;
      expect(Number.isFinite(v) && Number.isFinite(d)).toBe(true);
    }
    return { v, flips, maxLateDelta };
  }
  it('LSD 把打滑輪的轉速差壓下來且不抖；locked 兩輪幾乎同速', () => {
    // 這裡沒有引擎模型（固定輸入扭力），所以只比較左右轉速差，不比較加速
    const open = splitMu({ type: 'open', preload: 0, lockCoef: 0 });
    const lsd = splitMu({ type: 'lsd', preload: 50, lockCoef: 0.25 });
    const locked = splitMu({ type: 'locked', preload: 0, lockCoef: 0 });
    console.log(
      `第 2–3 秒最大左右轉速差 open ${open.maxLateDelta.toFixed(1)} / lsd
${lsd.maxLateDelta.toFixed(1)} / ` +
        `locked ${locked.maxLateDelta.toExponential(2)} rad/s；LSD 轉速差翻轉次數 ${lsd.flips}`,
    );
    expect(lsd.maxLateDelta).toBeLessThan(open.maxLateDelta);
    expect(locked.maxLateDelta).toBeLessThan(0.5);
    expect(lsd.flips).toBe(0);
    expect(locked.flips).toBeLessThan(5);
  });
});

describe('轉向輔助', () => {
  const p: SteeringAssistParams = {
    enabled: true,
    strength: 1,
    peakSlipAngleRad: 8 * DEG,
    targetRatio: 1.05,
    fadeStart: 5,
    fadeEnd: 10,
    countersteerEnabled: true,
    countersteerGain: 0.6,
    countersteerThresholdRad: 9 * DEG,
    countersteerMaxRad: 15 * DEG,
    maxWheelAngleRad: 33 * DEG,
  };
  it('高速打滿右轉時，輪角被限制在最佳滑動角窗口', () => {
    const out = updateSteeringAssist(p, {
      playerAngle: 20 * DEG,
      frontAxle: { lateral: 0, forward: 25 },
      rearAxle: { lateral: 0, forward: 25 },
    });
    expect(out.limited).toBe(true);
    expect(out.angle).toBeCloseTo(8.4 * DEG, 4);
  });
  it('低速不限制', () => {
    const out = updateSteeringAssist(p, {
      playerAngle: 30 * DEG,
      frontAxle: { lateral: 0, forward: 3 },
      rearAxle: { lateral: 0, forward: 3 },
    });
    expect(out.angle).toBeCloseTo(30 * DEG, 6);
```

```
  });
  it('窗口只限制幅度，不會把輪子轉到玩家沒打的那一側', () => {
    const out = updateSteeringAssist({ ...p, countersteerEnabled: false }, {
      playerAngle: 10 * DEG,
      frontAxle: { lateral: -8, forward: 20 },
      rearAxle: { lateral: 0, forward: 20 },
    });
    expect(out.angle).toBeGreaterThanOrEqual(0);
  });
  it('右彎甩尾（後軸往左滑）時自動往左反打', () => {
    const out = updateSteeringAssist(p, {
      playerAngle: 0,
      frontAxle: { lateral: 0, forward: 20 },
      rearAxle: { lateral: -6, forward: 20 },
    });
    expect(out.countersteer).toBeLessThan(0);
    expect(out.angle).toBeLessThan(0);
  });
});

describe('STM', () => {
  const p: StmParams = {
    enabled: true,
    minSpeed: 4.17,
    wheelbase: 2.65,
    kUs: 0.0025,
    muEstimate: 1.05,
    limitRatio: 1.0,
    deadband: 0.08,
    kp: 2500,
    ki: 4000,
    maxFrontBrake: 800,
    maxRearBrake: 400,
    throttleCutGain: 2,
    minThrottleScale: 0.2,
    filterTau: 0.05,
    suspendWithHandbrake: true,
  };
  it('右轉過度轉向 → 煞左前、降油門', () => {
    const st = createStmState();
    let out = updateStm(p, st, { forwardSpeed: 20, driverSteerAngle: 3 * DEG, yawRateRight: 0.9,
handbrake: 0, dt: DT });
    for (let i = 0; i < 30; i++)
      out = updateStm(p, st, { forwardSpeed: 20, driverSteerAngle: 3 * DEG, yawRateRight: 0.9,
handbrake: 0, dt: DT });
    expect(out.mode).toBe('oversteer');
    expect(out.brakeTorque[0]).toBeGreaterThan(100);
    expect(out.brakeTorque[1]).toBeCloseTo(0, 3);
    expect(out.throttleScale).toBeLessThan(0.9);
  });
  it('極限定圓（1.0 g）時 yaw 多出 10% 的正常波動不會觸發', () => {
    const v = 22.15;
    const r = (GRAVITY * 1.0 * 1.1) / v;
    for (const [ratio, expectMode] of [
      [1.0, 'none'],
      [0.85, 'oversteer'],
    ] as const) {
      const st = createStmState();
      const out = updateStm({ ...p, limitRatio: ratio }, st, { forwardSpeed: v, driverSteerAngle:
12 * DEG, yawRateRight: r, handbrake: 0, dt: DT });
      console.log(`STM limitRatio ${ratio}: ${out.mode}`);
      expect(out.mode).toBe(expectMode);
```

```
    }
  });
  it('拉手煞車時暫停，放開後 0.2 秒內恢復介入', () => {
    const st = createStmState();
    const base = { forwardSpeed: 20, driverSteerAngle: 3 * DEG, yawRateRight: 0.9, dt: DT };
    let out = updateStm(p, st, { ...base, handbrake: 1 });
    for (let i = 0; i < 30; i++) out = updateStm(p, st, { ...base, handbrake: 1 });
    expect(out.mode).toBe('none');
    expect(Math.max(...out.brakeTorque)).toBeLessThan(1);
    for (let i = 0; i < 24; i++) out = updateStm(p, st, { ...base, handbrake: 0 });
    expect(out.mode).toBe('oversteer');
    expect(out.brakeTorque[0]).toBeGreaterThan(100);
  });
  it('右轉轉向不足 → 輕煞右後', () => {
    const st = createStmState();
    let out = updateStm(p, st, { forwardSpeed: 15, driverSteerAngle: 8 * DEG, yawRateRight: 0.1,
handbrake: 0, dt: DT });
    for (let i = 0; i < 30; i++)
      out = updateStm(p, st, { forwardSpeed: 15, driverSteerAngle: 8 * DEG, yawRateRight: 0.1,
handbrake: 0, dt: DT });
    expect(out.mode).toBe('understeer');
    expect(out.brakeTorque[3]).toBeGreaterThan(50);
    expect(out.brakeTorque[3]).toBeLessThanOrEqual(400);
  });
});
```

tests/longitudinal.test.ts（整車縱向估算測試）

```
import { describe, expect, it } from 'vitest';
import { DEG, GRAVITY } from '../src/vehicle/math';
import { createTireState, type TireParams } from '../src/vehicle/Tire';
import { stepWheel, type WheelState } from '../src/vehicle/Wheel';
import { stepDrivenAxle } from '../src/vehicle/Drivetrain';
import { createTcsState, updateTcs } from '../src/vehicle/assists/tcs';
import { createAbsState, updateAbs } from '../src/vehicle/assists/abs';

/**
 * 整車縱向估算：直線、準靜態前後重量轉移、預設 preset 的引擎/齒比/空力/煞車。
 * 不含懸吊動態、離合器細節與 Rapier，只用來檢查「預設參數和驗收目標是否一致」。
 */
const tire: TireParams = {
  peakSlipRatio: 0.1, cLong: 1.5, muLong: 1.1, peakSlipAngleRad: 8 * DEG, cLat: 1.3, muLat: 1.05,
  loadSensitivity: 0.1, nominalLoad: 3300, relaxLenLat: 0.35, relaxLenLong: 0.1,
};
const wheel = { radius: 0.33, inertia: 1.0 };
const M = 1350, L = 2.65, A = 1.27, B = 1.38, H = 0.5;
const GEARS = [3.4, 2.2, 1.6, 1.25, 1.0, 0.82], FINAL = 3.7, EFF = 0.9;
const CURVE: [number, number][] = [[1000, 200], [2000, 300], [3000, 350], [4000, 350], [5000,
340], [6000, 310], [7000, 260]];
const torqueAt = (rpm: number) => {
  if (rpm <= CURVE[0]![0]) return CURVE[0]![1];
  for (let i = 1; i < CURVE.length; i++) {
    const [r1, t1] = CURVE[i]!, [r0, t0] = CURVE[i - 1]!;
    if (rpm <= r1) return t0 + ((t1 - t0) * (rpm - r0)) / (r1 - r0);
  }
  return CURVE[CURVE.length - 1]![1];
};
const DT = 1 / 120, SUB = 4;
const tcsP = { enabled: true, peakSlipRatio: 0.1, engageRatio: 1.15, targetRatio: 1, kp: 0.25,
ki: 5, recoverRate: 3, disableWithHandbrake: true };
```

```
const absP = { enabled: true, peakSlipRatio: 0.1, engageRatio: 1.15, targetRatio: 1,
releaseRatio: 0.5, kp: 1.5, ki: 20, minPressure: 0.05, minSpeed: 1.39, hudPulseHz: 15 };

function sim(mode: 'accel' | 'brake', seconds: number) {
  const w: WheelState[] = [0, 1, 2, 3].map(() => ({ omega: 0, tire: createTireState() }));
  let v = mode === 'brake' ? 100 / 3.6 : 0;
  w.forEach((s) => (s.omega = v / wheel.radius));
  let x = 0, a = 0, gear = 0, shiftTimer = 0, sinceShift = 1, t = 0;
  let t100: number | null = null, stopDist: number | null = null, vmax = 0;
  const tcs = createTcsState();
  const abs = [0, 1, 2, 3].map(() => createAbsState());

  for (let i = 0; i < seconds / DT; i++, t += DT) {
    const down = 0.5 * 1.225 * 0.3 * v * v;
    const fzF = (M * GRAVITY * B - M * a * H) / L / 2 + (down * 0.45) / 2;
    const fzR = (M * GRAVITY * A + M * a * H) / L / 2 + (down * 0.55) / 2;
    const rr = (fz: number) => 0.012 * fz * wheel.radius;
    let fxTotal = 0;

    if (mode === 'accel') {
      const ratio = GEARS[gear]! * FINAL;
      const wheelRpm = (((w[2]!.omega + w[3]!.omega) / 2) * ratio * 60) / (2 * Math.PI);
      // 起步半離合: 引擎維持在 3000 rpm 附近輸出; 排檔判斷一律用車輪換算轉速
      const engineRpm = Math.max(wheelRpm, 3000);
      sinceShift += DT;
      if (wheelRpm > 6500 && gear < GEARS.length - 1 && sinceShift > 0.5) {
        gear++; shiftTimer = 0.15; sinceShift = 0;
      }
      shiftTimer = Math.max(0, shiftTimer - DT);
      const maxSlip = Math.max(w[2]!.tire.slipRatio, w[3]!.tire.slipRatio);
      const scale = updateTcs(tcsP, tcs, maxSlip, 0, DT);
      const cut = shiftTimer > 0 || engineRpm > 7000 ? 0 : 1;
      const axleT = torqueAt(engineRpm) * cut * scale * GEARS[gear]! * FINAL * EFF;
      const iEff = wheelRpm >= 1500 ? wheel.inertia + (0.2 * (GEARS[gear]! * FINAL) ** 2) / 2 :
wheel.inertia;
      const common = { vy: 0, surfaceGrip: 1, brakeTorque: 0, inertiaEff: iEff };
      const [l, r] = stepDrivenAxle(tire, wheel, [w[2]!, w[3]!],
        [{ ...common, vx: v, fz: fzR, resistTorque: rr(fzR) }, { ...common, vx: v, fz: fzR,
resistTorque: rr(fzR) }],
        { type: 'lsd', preload: 50, lockCoef: 0.25 }, axleT, DT, SUB);
      fxTotal += l.fx + r.fx;
      for (const k of [0, 1]) {
        fxTotal += stepWheel(tire, wheel, w[k]!, { vx: v, vy: 0, fz: fzF, surfaceGrip: 1,
driveTorque: 0, brakeTorque: 0, resistTorque: rr(fzF), inertiaEff: wheel.inertia, dt: DT,
substeps: SUB }).fx;
      }
    } else {
      for (let k = 0; k < 4; k++) {
        const maxT = k < 2 ? 2200 : 900;
        const fz = k < 2 ? fzF : fzR;
        const p = updateAbs(absP, abs[k]!, w[k]!.tire.slipRatio, v, 1, DT).pressureScale;
        fxTotal += stepWheel(tire, wheel, w[k]!, { vx: v, vy: 0, fz, surfaceGrip: 1, driveTorque:
0, brakeTorque: maxT * p, resistTorque: rr(fz), inertiaEff: wheel.inertia, dt: DT, substeps: SUB
}).fx;
      }
    }

    const drag = 0.5 * 1.225 * 0.72 * v * Math.abs(v);
    a = (fxTotal - drag) / M;
    v += a * DT;
    x += v * DT;
    vmax = Math.max(vmax, v);
```

```
    if (mode === 'accel' && t100 === null && v >= 100 / 3.6) t100 = t;
    if (mode === 'brake' && stopDist === null && v <= 0.1) { stopDist = x; break; }
  }
  return { t100, stopDist, vmaxKmh: vmax * 3.6 };
}

describe('整車縱向估算（預設 preset 與驗收目標是否一致）', () => {
  it('0→100、極速、100→0', () => {
    const acc = sim('accel', 60);
    const brk = sim('brake', 10);
    console.log(`0→100 ${acc.t100?.toFixed(2)} s; 60 秒內最高 ${acc.vmaxKmh.toFixed(0)} km/h; 100→0
${brk.stopDist?.toFixed(1)} m`);
    expect(acc.t100!).toBeGreaterThanOrEqual(6.0);
    expect(acc.t100!).toBeLessThanOrEqual(7.5);
    expect(acc.vmaxKmh).toBeGreaterThanOrEqual(230);
    expect(acc.vmaxKmh).toBeLessThanOrEqual(260);
    expect(brk.stopDist!).toBeGreaterThanOrEqual(36);
    expect(brk.stopDist!).toBeLessThanOrEqual(42);
  });
});
```

六、已知限制

1. 上面的驗證是「四分之一車」、純函式測試，以及只有直線的整車縱向估算，**沒有包含側向動態、懸吊與 Rapier 剛體**。定圓、繞錐、甩尾手感，要等 Phase 7 的 handling-report 才能驗證。
2. 預設參數是依照一般 1350 kg 後驅轎車估算的起點，0→100、極速、煞車距離都是推算值，實車開起來仍需要調校。
3. 決定性只保證同一台機器、同一個瀏覽器。多人連線請用狀態同步，不要依賴各端重算出相同結果。
4. 方向盤力回饋在瀏覽器支援度有限，規格中列為不做。
5. 之後加入真實地景、使用 floating origin（世界原點平移）時，駐車錨點是世界座標，必須跟著一起平移。